arXiv:2611.12885v1 [cs.CV] 9 Nov 2026

RoboScript

World In-Context Learning for Agentic Robot Manipulation

Heng Fang[1], **Zehui Guo**[1], **Yuehan Zhang**[1], **Mingyang Du**[1,2], **Dingkang Liang**[1,2], **Xiang Bai**[1]

[1]Huazhong University of Science and Technology, [2]ArcheOne AI

{hengfang,dkliang,xbai}@hust.edu.cn

Abstract

Trained action policies write manipulation skill into network weights, tying what is learned to the distribution it is learned from. In-context learning (ICL) avoids this coupling but has not matched trained policies, because a demonstration recorded in another scene carries no feedback from the world the robot currently faces. We observe that what an agent learns in context is semantic, recording task steps and object roles rather than visual appearance, so experience gathered in one world transfers to another. Based on this insight, we propose World In-Context Learning (WICL), a paradigm in which a frozen model starts from a demonstration, rehearses the task in a world such as a simulator, and writes the feedback into its context rather than its weights. We build RoboScript, a modular infrastructure that implements WICL for agentic manipulation. From a single demonstration, RoboScript learns each task through rehearsal in simulation, freezes the learned knowledge as a Task Memory, and deploys the frozen memory on new scenes with data isolation that ensures every deployment input is available on a real robot. On all 50 tasks of RoboTwin 2.0, RoboScript reaches 83.4% mean success across clean and randomized settings with one demonstration per task and no gradient updates, exceeding the strongest trained action policy by over 20 points. The method generalizes across visual distributions, improves through reflection, scales with the foundation model, and transfers to a physical dual-arm robot through cross-embodiment memory adaptation. With a world in the loop, a robot can learn a manipulation task in its context as well as it learns one in its weights.

Code: https://github.com/ArcheOne-AI/RoboScript
Project page: https://ArcheOne-AI.github.io/RoboScript

1 Introduction

In-context learning (ICL) is a defining capability of foundation models. A model with frozen weights adapts to a new task from what its context shows, without any gradient update [5, 14]. Robot manipulation has long followed a different path. The dominant approach trains an action policy, either a vision-language-action (VLA) model [2, 3, 41, 55, 59, 104] or a world action model (WAM) [12, 86], and writes manipulation skill into network weights. This approach has made clear progress, but it ties what is learned to the distribution it is learned from. FastWAM [89], a widely adopted WAM trained on clean demonstrations of RoboTwin 2.0, reaches 77.8% mean success on clean scenes but drops to 1.9% under domain randomization [11]. Every new task or distribution requires new data and new training. ICL has recently emerged as a promising alternative that avoids this coupling. Recent models can perform a new manipulation task from a single demonstration given in their context, without any weight update [26, 68, 97]. This progress raises the question we study. Can a robot learn a manipulation task in its context as well as it learns one in its weights?

The in-context route avoids this coupling, but it has not matched trained policies, because its context carries

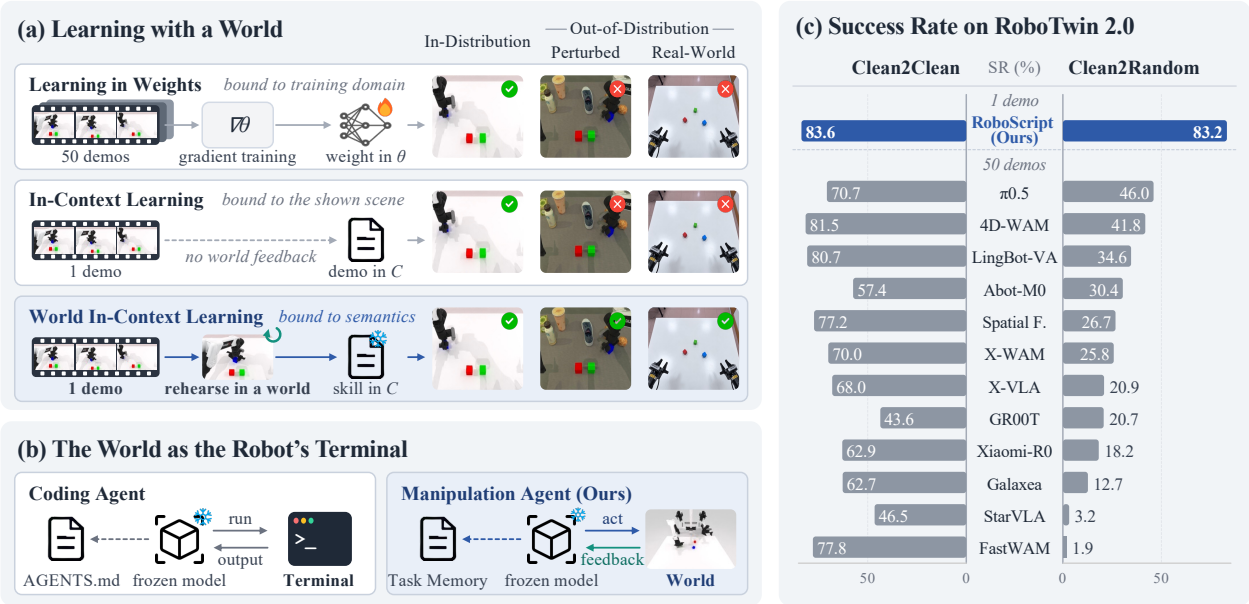


Figure 1 (a) Three paradigms for learning manipulation. Skill in weights is bound to the training domain. ICL avoids retraining but is bound to the shown scene. WICL rehearses in a world and stores semantic knowledge that transfers across settings. (b) RoboScript maps the coding-agent stack to robot manipulation, with the world replacing the terminal. (c) Success rate on all 50 RoboTwin 2.0 tasks with one demonstration and no gradient updates.

no feedback from the world. A demonstration is recorded in another scene and cannot tell the robot what will happen on the table in front of it. A frontier model (e.g. GPT-6 Astra) did not improve when given a demonstration from a different layout in its prompt, although some of its traces show it correcting its actions after observing their outcomes [92]. Code as Policies and its successors can control a robot through APIs, but without world feedback their errors accumulate [32, 53, 67, 88]. Coding agents show what is missing. They learn a new repository by running code in a terminal and reading what comes back [76, 83]. The feedback loop they rely on has no counterpart in robot manipulation.

Our key observation is that what an agent learns in context is semantic. It records object roles, task steps, and what went wrong, not the visual appearance of the world. Experience gathered in one world therefore carries over to another, as long as the two worlds agree on the semantics of the task. A cheap world that is faithful to the task, such as a simulator, a learned world model, or a latent model of the scene, is enough to learn in. Rehearsal in such a world is also safe, unlike acting directly on real hardware, where unsafe actions have damaged equipment [92].

We formalize this insight as **World In-Context Learning** (WICL), a learning paradigm that extends in-context learning with feedback from a world, as illustrated in Fig. 1a. A WICL learner keeps its weights frozen, starts from a demonstration, rehearses the task in a world, and distills the feedback into its context. The resulting knowledge is semantic and transfers from the rehearsal world to the deployment world without retraining. After rehearsal, the context is frozen and evaluated on instances that were never rehearsed, so the result measures generalization, not memorization. The context is readable text, which allows the agent to review its own evaluations and rewrite what it has learned, enabling recursive self-improvement (RSI) without gradient updates. The world makes every round of improvement safe. Coding agents have shown that this loop already succeeds in software engineering, where the terminal serves as the world [76, 83]. Now that foundation models have reached this capability, a simulator can serve the same role for a robot.

We build **RoboScript**, a modular infrastructure for agent-centric robot manipulation. Fig. 1b illustrates the analogy and Fig. 1c reports the main results. In this paper, we implement WICL on RoboScript with a lifecycle that the agent carries out entirely without gradient updates. The foundation model first builds skills for a new environment on shared entry points. It then learns each task from a single demonstration and from rehearsal on training scenes, and freezes what it learns as a Task Memory. At deployment, it acts with the

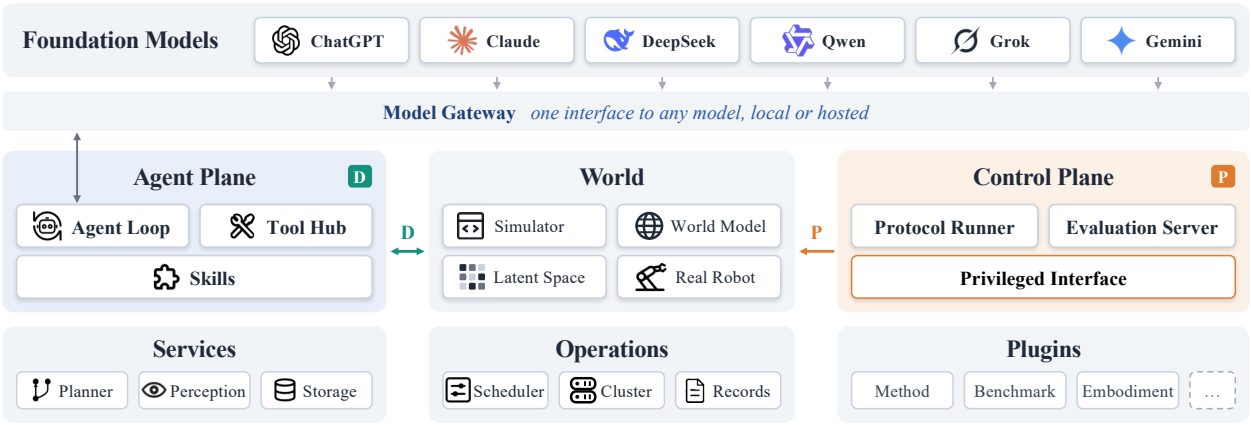


Figure 2 Infrastructure of RoboScript. Every component runs as an independent service, and any method, benchmark, embodiment, or extension plugs in as a replaceable module. All model calls pass through one gateway. The agent plane reaches the world only through the deployable interface (D), and only the control plane accesses privileged data (P).

frozen memory on new scenes, anticipating how the task should unfold and recovering from failures. After evaluation, a reflection agent reviews the records and writes guidance for the next round, realizing RSI over several iterations. Data isolation is enforced by construction. Every acting agent, whether in rehearsal or in deployment, observes only camera images with depth and proprioception, the same inputs available on a real robot. Privileged simulator information serves only as supervision during learning, and a memory linter rejects coordinates, ground-truth terms, and seed identifiers. This strict separation ensures that what is learned in simulation is directly deployable on a physical robot. As shown in Fig. 2, the method, environment, embodiment, and foundation model are modular like Lego blocks. Each runs as an independent service and can be replaced, making RoboScript a reusable platform for agentic manipulation.

We conduct extensive experiments and present the main results on all 50 tasks of RoboTwin 2.0, leading to five findings. 1) One demonstration with rehearsal is sufficient. With one clean demonstration per task, RoboScript reaches 83.4% mean success across the clean and randomized settings without gradient updates, whereas the strongest trained action policy, trained with 50 demonstrations, reaches 61.7%. 2) What is learned in context generalizes across distributions. Rehearsing only on clean scenes, RoboScript maintains comparable success under domain randomization, whereas trained action policies retain at most 65.1% of their clean success. 3) Reflection improves the frozen memory. Each round of reflection rescues more episodes than it breaks on difficult tasks. 4) The framework improves with the foundation model. Without any change to code, prompts, or protocol, GPT-6.1-Sol reaches 83.6/83.2% on clean/randomized scenes, GPT-5.6-Luna reaches 79.8/76.0%, and even DeepSeek-V4.1-Flash, a lightweight model, reaches 77.8/78.4%, surpassing the strongest trained action policy. 5) Task Memories transfer across embodiments. With a lightweight adapter that rewrites an existing memory for a new robot, RoboScript deploys on a physical dual-arm robot without retraining or re-rehearsing. These findings jointly show that with a world in the loop, a robot can learn a manipulation task in its context as well as it learns one in its weights.

Our contributions are as follows.

- We formulate WICL, a learning paradigm that extends in-context learning with feedback from a world. WICL decouples task skill from network weights and opens a path toward recursive self-improvement without retraining. We explain why the resulting knowledge transfers across worlds.
- We build a modular infrastructure for agent-centric manipulation with strict data isolation that ensures deployability on real hardware. We implement WICL on RoboScript with a lifecycle covering skill building, task learning, deployment, and recursive self-improvement, all without gradient updates.
- We conduct extensive experiments with one demonstration per task and report results on RoboTwin 2.0, across distributions, over multiple rounds of self-improvement, with three foundation models, and on a physical dual-arm robot via cross-embodiment memory transfer, achieving state-of-the-art performance.

2 Related Work

Action Policies for Manipulation. The dominant approach to robot manipulation trains an action policy and stores skill in network weights. Vision-language-action (VLA) models learn visuomotor skills end to end from demonstrations, typically after pretraining on large cross-embodiment datasets [2, 3, 6, 40, 41, 55, 59, 61, 81, 104], with the VLA paradigm also advancing autonomous driving [21, 22, 100] and benefiting from efficient 3D representations [49, 50]. World action models (WAMs) generate future observations or latent plans and derive actions from them [9, 16, 42, 80, 86], and driving world models jointly predict future scenes and 3D structure [52, 98, 99]. Reinforcement learning optimizes a policy from simulator feedback [46, 62, 95] or from real-robot trials [39, 56, 57]. All three families have advanced rapidly, yet they share a structural limitation. Because skill resides in weights, trained policies degrade under distribution shift [11, 18, 25, 90, 101], and each new task or domain requires additional data and gradient updates. WICL offers an alternative route. It keeps the weights frozen and stores what it learns in context, so a new task requires rehearsal rather than retraining.

In-Context Manipulation. In robotics, in-context learning has mainly taken the form of imitation. One-shot and few-shot methods interpret a demonstration given at test time [17, 20, 24, 36, 60, 73], while other methods pass keypoints or keyframes to a frozen model [13, 63, 87] or retrieve demonstrations for a trained policy [69, 82]. Recent work has shown that in-context manipulation from a single demonstration is increasingly effective [26, 68, 97]. A parallel line uses frozen models as agents. Training-free systems compose APIs, value maps, or keypoints [19, 23, 32, 34, 53], while hierarchical systems place a language model above a trained policy [10, 15, 33, 35, 51, 65]. Embodied harnesses add tools, verification, and memory [27, 31, 54, 75, 93]. These methods place demonstrations, instructions, or tool descriptions in the context, but none distills experience from a world into a reusable memory. Among these, HarnessVLA learns from interaction but depends on a trained policy for contact [93], and FAEA reasons iteratively within an episode but reads privileged simulator state [72]. In contrast, RoboScript removes the trained policy from the loop and enforces data isolation by construction. WICL adds the missing feedback. Where ICL gives the context a demonstration, WICL also gives it outcomes from a world, and the acting agent perceives the scene through cameras only.

Self-Improving Agents and World Feedback. Language model agents can improve from interaction while their weights stay frozen. Reflexion stores reflections on failed trials, Voyager grows a library of code skills, and ExpeL extracts insights from past trajectories [66, 74, 94]. Later work rewrites the context itself [1, 77, 91]. Coding agents solve real software tasks through the same loop [76, 83], though they learn within a single episode and do not distill experience across episodes. These methods work in text and code. WICL brings the same principle to physical manipulation, where the learned knowledge is semantic and transfers across worlds. In-context reinforcement learning trains a model on prior interaction data so that it improves within its context at test time [43, 44], but it requires this supervised pretraining. In manipulation, self-improving systems collect practice data and retrain their policies [4, 64, 71, 102], and frozen language models have used simulator feedback to write reward functions that then train policies [58]. All of these use world feedback to update weights. World models have been used to learn behaviors in imagination [30, 37, 78], to train policies with reinforcement learning [103], to evaluate existing policies [29, 48]. WICL uses the world differently. It feeds outcomes back into the context with frozen weights, distills experience across episodes into a reusable memory, and extends the loop to recursive self-improvement without gradient updates.

3 World In-Context Learning

Formulation. WICL extends in-context learning with feedback from a world, in the same way that reinforcement learning extends imitation learning. Let π_θ be a foundation model with frozen weights θ that acts conditioned on an observation o and a context C. A task is specified by a demonstration d and a set of instances, split into a rehearsal set $\mathcal{E}_{\text{reh}}$ and an evaluation set $\mathcal{E}_{\text{test}}$ with $\mathcal{E}_{\text{reh}} \cap \mathcal{E}_{\text{test}} = \emptyset$. A world W takes an instance e and a policy, and returns a trajectory τ together with feedback r. Learning starts from a context built from the demonstration and alternates between rehearsal and revision, as shown in Fig. 3a,

$$C_0 = \mathcal{I}_\theta(d), \qquad (\tau_k, r_k) \sim W\big(\pi_\theta(\cdot \mid o, C_k),\, e_k\big), \qquad C_{k+1} = \mathcal{U}_\theta(C_k, \tau_k, r_k), \tag{1}$$

where $e_k \in \mathcal{E}_{\text{reh}}$ and W supplies the observation o at each step. The same frozen model performs the initialization $\mathcal{I}_\theta$ and the update $\mathcal{U}_\theta$, so learning changes only the context. Learning stops when a convergence

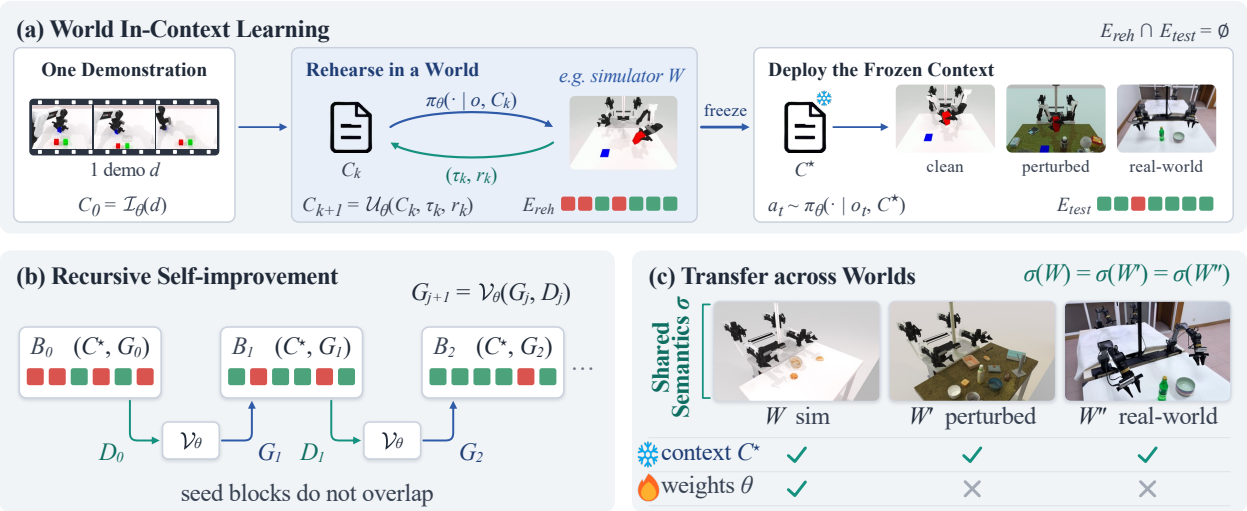


Figure 3 (a) The WICL pipeline. From one demonstration, the agent rehearses in a world, freezes the context, and deploys it on unseen instances. (b) Recursive self-improvement over disjoint evaluation blocks with frozen weights. (c) Context knowledge refers to shared semantics σ and transfers across worlds, whereas trained weights do not.

rule is met, and the final context $C^\star$ is frozen. The learner then acts on new instances with the frozen context and receives no feedback from the world,

$$a_t \sim \pi_\theta(\cdot \mid o_t, C^\star), \qquad e \in \mathcal{E}_{\text{test}}. \tag{2}$$

Since no evaluation instance is rehearsed, $C^\star$ is tested on generalization, not memorization. In practice, C is natural-language text that the model reads at every step.

Transfer across Worlds. The knowledge in $C^\star$ transfers across worlds because it is semantic, as illustrated in Fig. 3c. Let $\sigma(e)$ denote the semantic content of an instance, namely the objects present, their roles, the instruction, and the relative geometry on which the task depends. The context $C^\star$ refers only to σ and to quantities that are measured at execution time. Suppose a second world W' agrees with W on the semantic content of its instances and on how behaviors change it, that is, $\sigma(W) = \sigma(W')$. Then the knowledge rehearsed in W still applies in W', even if W' differs in appearance, lighting, or physics parameters. Weights trained in W behave differently. They absorb every regularity of W that helps fit the training data, including regularities that W' does not share. This asymmetry is why WICL can learn in a cheap world and deploy in another, whether that is a randomized simulator or a real robot.

Recursive Self-Improvement. The learner can also improve from its own evaluations, as shown in Fig. 3b. Let $\mathcal{B}_0, \mathcal{B}_1, \ldots$ be disjoint blocks of evaluation instances and let $G_0 = \emptyset$. After evaluating on $\mathcal{B}_j$ with context $(C^\star, G_j)$, the learner reflects on the resulting records D_j and writes guidance for the next block,

$$G_{j+1} = \mathcal{V}_\theta(G_j, D_j). \tag{3}$$

The records D_j contain only what a deployed system would observe, together with whether each episode succeeded. They do not say why an episode failed, so the learner must diagnose the causes itself. Each block is evaluated before the learner sees its outcomes, and the next block is always unseen. This realizes recursive self-improvement with frozen weights.

What a World Must Provide. A world is useful for WICL if it meets three conditions. It must be faithful to the task, so that behaviors change the semantic content σ in the world as they would in the target environment. It must be resettable, so that a behavior can be retried and compared. And it must keep rehearsal instances apart from evaluation instances. When no world is queried, WICL reduces to ICL, with $C^\star = C_0 = \mathcal{I}_\theta(d)$. In this paper, the world is the RoboTwin 2.0 simulator [11], and Sec. 4 describes our implementation.

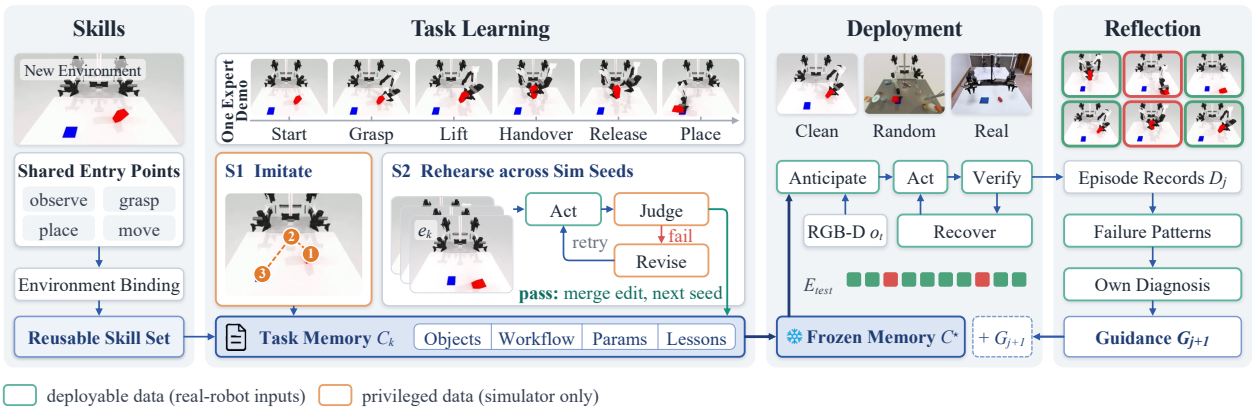


Figure 4 The four stages of RoboScript and the data tier of each role. The agent builds reusable skills, learns a Task Memory from one demonstration through imitation (S1) and rehearsal (S2), deploys the frozen memory with anticipation and recovery, and improves through reflection over disjoint evaluation blocks. Green borders mark deployable data (real-robot inputs), and orange borders mark privileged data (simulator only).

4 RoboScript

RoboScript implements WICL for robot manipulation. Fig. 4 shows the pipeline together with the data tier of each role. The world W is a simulator, the demonstration d is a single expert trajectory, and the learned context $C^\star$ is a Task Memory. The agent builds its action interface (Sec. 4.1), learns a Task Memory from one demonstration (Sec. 4.2), and deploys it on unseen instances (Sec. 4.3). It can then improve the frozen memory through reflection (Sec. 4.4). Infrastructure and protocol parameters are given in the appendix.

4.1 Perception and action

The agent perceives and acts only through tools, and the same tools run in simulation and on a real robot.

Geometric-first perception. The agent observes RGB-D images from head and wrist cameras together with proprioception. Object detection starts from the depth channel. A height-map above the workspace produces connected-component proposals, each lifted to a 3D bounding volume with an attribute card that records its center, extent, and orientation. Object identity is resolved by a three-way vote among Set-of-Mark annotation, a grounding model, and a VLM. Once identified at the start of an episode, an object is tracked by its grasp relation to the end effector instead of re-detected after every action. Part-level localization follows the same pipeline on a cropped region.

Action interface with receipts. The agent acts through a three-layer API. Shared entry points for grasping, placing, corrective motions, reachability checks, and coordinated bimanual motion are environment-independent. An environment binding discovered by automated probes supplies coordinate frames, joint limits, table height, and gripper characteristics. Before learning, the agent builds environment-specific skills on these two layers. It segments each demonstration into action primitives, identifies which segments the existing entries cannot explain, and hands each gap to a coding agent that writes a skill. A driver validates each skill by running it on the demonstration scene and checking that it transfers to other layouts without encoding any absolute position. The loop repeats until coverage stops improving, and the final skill set is frozen before task learning.

Every action returns a receipt reporting the commanded pose, the achieved pose, the measured gripper opening, and whether an object is held. Receipts make the feedback r in Eq. (1) concrete. Agent distinguishes a command that was executed from an effect that was achieved to decide its next step. Receipts are derived from proprioception and the motion planner, not from simulator state, so they match on a real robot.

4.2 Task learning from one demonstration

The agent learns a Task Memory $C^\star$ from a single demonstration d in two stages that together realize Eq. (1). Both stages rehearse in the world, differing only in which instances the agent faces. As shown in Fig. 4, border

color marks the data tier of each role.

Task Memory as semantic knowledge. The Task Memory is a structured Markdown document that describes the roles of the objects and how to find them, the workflow of each stage, specifications for grasping and placing, learned parameters, and lessons from past attempts. GraspSpec and PlaceSpec are relative specifications expressed in terms of perceived object parts, such as "approach the handle from the left and grasp at its midpoint." They never contain positions in a particular scene. This representation is the concrete form of the semantic knowledge in Sec. 3. The memory refers only to object roles and relative geometry and therefore applies to any scene that contains the same objects in reachable positions. A memory linter enforces this property by rejecting coordinates, ground-truth terms, and instance identifiers.

Imitation (S1). Imitation starts from the initialization $C_0 = \mathcal{I}_\theta(d)$. A reader role studies the demonstration and writes an initial Task Memory. An acting role then practices the task on the demonstration instance, and a reviewing role revises the memory after each attempt until the agent succeeds repeatedly.

Rehearsal (S2). Rehearsal extends this to new training instances, realizing $C_{k+1} = \mathcal{U}_\theta(C_k, \tau_k, r_k)$. The agent practices on one instance after another, and only its first attempt on each instance counts. After a failure, the reviewing role receives a ground-truth diagnosis and revises a cached copy of the memory. The revision is merged into the shared memory only if it makes that instance succeed. Edits are typed and subject to a budget that decreases with experience, so the memory stabilizes as the agent practices. Rehearsal stops when enough recent first attempts succeed, and the memory is frozen after passing the linter.

Data isolation during learning. As Fig. 4 indicates, every acting agent observes only deployable data, the same inputs available on a real robot. Privileged data enters only as supervision. In S1, the reader uses it to understand the demonstration. In S2, the reviewer receives a ground-truth diagnosis of each failure. What this supervision teaches reaches the acting agent only through the Task Memory, and the linter ensures that the memory holds semantic knowledge, not simulator facts. The actor therefore runs entirely on data that a real robot can provide and never accesses any simulator-specific privileged information. This isolation ensures that the learned memory is not specific to any simulator.

4.3 Deployment with anticipation and recovery

At deployment, the agent acts with the frozen memory $C^\star$ on instances it has never seen and receives no feedback r from the world, as defined by Eq. (2). It runs under the official protocol of the benchmark with unseen instructions. As Fig. 4 shows, the deployed actor uses only deployable data.

Scene anticipation. Before manipulating an object, the agent maps the demonstration's semantic blueprint onto the current scene. It identifies which object plays each role in the memory, checks which arm can reach each grasp and placement, and verifies that the planned sequence is feasible. This step may call the motion planner but never receives feedback about success and uses only deployable data.

Failure recovery without reset. A failed action does not end the episode. The agent moves to a safe pose, observes the scene again, and continues from the current state without resetting the environment, bounded only by the official step limit. The agent declares the task done only after its perception confirms that the scene matches the final state of the demonstration. The simulator's success check is the final judge, so a wrong declaration never counts as a success. On a real robot, the same procedure runs under human supervision.

4.4 Recursive self-improvement

The reflection loop realizes the recursive self-improvement defined by Eq. (3). After an evaluation block, a reflection agent reads the episode records, which contain only what a deployed robot would observe together with whether each episode succeeded. It identifies failure patterns and their causes and writes a guidance document for the next block. Since the blocks never overlap, each round tests whether the guidance generalizes to unseen instances. Repeating this loop over several rounds lets the agent improve without gradient updates or privileged data. No instance is ever revisited, so the improvement measures generalization, not memorization.

Table 1 Main results on RoboTwin 2.0. Mean success rate (%) over all 50 tasks. Retention is the randomized success divided by the clean success. Trained marks whether a method uses a trained action policy, and Agent marks whether it uses an agent. Trained action policies are trained jointly on all tasks with 50 clean demonstrations per task. †Randomized setting only, five seeds per task. N is unreported fine-tuning demonstrations for LingBot-VLA [79].

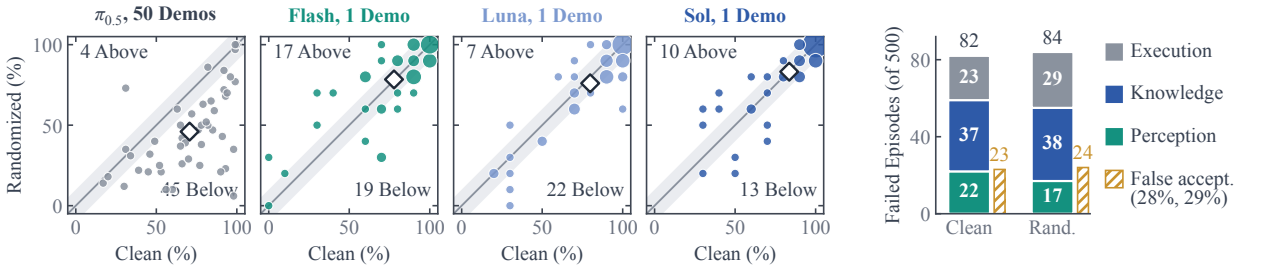
Method	Demos per task	Trained	Agent	Success Rate (%)			Retention (%)
				Clean	Rand.	Avg.	
Trained action policies							
Vision-language-action models (VLAs)							
StarVLA [70]	50	✓	✗	46.5	3.2	24.8	6.8
GR00T-N1.7 [59]	50	✓	✗	43.6	20.7	32.2	47.5
GalaxeaVLA [38]	50	✓	✗	62.7	12.7	37.7	20.3
Xiaomi-Robotics-0 [8]	50	✓	✗	62.9	18.2	40.6	28.9
Abot-M0 [85]	50	✓	✗	57.4	30.4	43.9	52.9
X-VLA [96]	50	✓	✗	68.0	20.9	44.5	30.7
Spatial Forcing [45]	50	✓	✗	77.2	26.7	52.0	34.6
$\pi_{0.5}$ [2]	50	✓	✗	70.7	46.0	58.4	65.1
World action models (WAMs)							
AHA-WAM [7]	50	✓	✗	64.3	3.2	33.8	5.0
FastWAM [89]	50	✓	✗	77.8	1.9	39.9	2.4
X-WAM [28]	50	✓	✗	70.0	25.8	47.9	36.9
LingBot-VA [47]	50	✓	✗	80.7	34.6	57.7	42.9
4D-WAM [84]	50	✓	✗	<u>81.5</u>	41.8	61.7	51.3
Agents with trained policies							
HarnessVLA (frozen base) [93]†	N	✓	✗	–	50.4	–	–
HarnessVLA (Codex) [93]†	$N+1$	✓	✓	–	58.0	–	–
HarnessVLA (Claude Code) [93]†	$N+1$	✓	✓	–	58.4	–	–
Action-policy-free agents							
RoboScript (DeepSeek-V4.1-Flash)	1	✗	✓	77.8	<u>78.4</u>	<u>78.1</u>	100.8
RoboScript (GPT-5.6-Luna)	1	✗	✓	79.8	76.0	77.9	95.2
RoboScript (GPT-6.1-Sol)	1	✗	✓	**83.6**	**83.2**	**83.4**	99.5

5 Experiments

We conduct extensive experiments in simulation and on real hardware, organizing the evaluation around six questions. Can one demonstration with rehearsal compete with policies trained on many demonstrations (Sec. 5.2)? Why does success hold under domain randomization (Sec. 5.3)? Where do the gains come from (Sec. 5.4)? Can the agent improve from its own evaluations (Sec. 5.5)? What does it cost (Sec. 5.6)? Does the learned knowledge transfer across embodiments to a real robot (Sec. 5.7)?

5.1 Experimental Setup

Benchmark and protocol. RoboTwin 2.0 [11] defines 50 dual-arm tasks in a clean setting (fixed background) and a randomized setting (randomized textures, lighting, table height, and distractors). The agent first builds its skill set across all 50 tasks, then learns each task from one automatically selected clean demonstration, with rehearsal on clean scenes only. The frozen Task Memory is evaluated in both settings with the official server, unseen instructions, and the official step limit. Each task is evaluated on 10 episodes per setting (1,000 episodes total), as each episode requires dozens of model calls. We report success rate averaged over all tasks, the mean of the two settings, and retention (randomized divided by clean).



(a) Per-Task Success, Clean vs. Randomized **(b) Failure Causes, Sol**

Figure 5 Success under randomization. (a) Per-task success. Dot area counts tasks at one point, the band marks a change within 10 points, and the diamond marks the mean. (b) Failed episodes of GPT-6.1-Sol by physical root cause. Hatched bars count false acceptances, which cut across the causes.

Baselines. Trained action policies (VLAs and WAMs) are trained on all 50 tasks with 50 clean demonstrations per task and evaluated on 100 episodes. The agent-with-policy baseline is HarnessVLA [93], which orchestrates a fine-tuned VLA with a coding agent. All baseline results come from the official leaderboard [11] or published reports, excluding policies trained on randomized data.

Foundation models and model service. We run RoboScript with GPT-6.1-Sol, GPT-5.6-Luna, and DeepSeek-V4.1-Flash with identical code, prompts, and protocol. API service quality varies over time and can shift success rates by several points between sessions. For the ablation study (Sec. 5.4), each group of variants and its control share one fixed endpoint and the same seeds, so within-group comparisons are paired, though absolute levels may differ across groups. Further settings are in Sec. C.

5.2 Main Results

Tab. 1 compares RoboScript with trained action policies and agents with trained policies on all 50 tasks.

One demonstration with rehearsal is sufficient. With one demonstration per task and no gradient update, RoboScript (GPT-6.1-Sol) reaches a mean of 83.4% (83.6 clean, 83.2 randomized), with a 95% Wilson interval of 81.0 to 85.6. The strongest trained policy, 4D-WAM trained on 50 demonstrations per task, reaches 61.7% (81.5 clean, 41.8 randomized). The two are comparable in the clean setting, yet the mean differs by 21.7 points because trained policies lose most of their success under randomization. These gains come from a single demonstration and rehearsal in a simulator, without any training data, gradient computation, or access to the randomized distribution.

What is learned in context holds across distributions. The three foundation models retain 95.2% to 100.8% of their clean success under randomization, whereas trained policies retain at most 65.1%. The Task Memory stores semantic knowledge that does not depend on visual appearance, so it applies equally to both settings. Even though the agent rehearses only on clean scenes and never observes any randomized instance, RoboScript exceeds HarnessVLA (58.4% randomized), which fine-tunes its base policy on RoboTwin data, by 24.8 points. Sec. 5.3 analyzes where the remaining failures come from.

The advantage does not depend on the strongest foundation model. DeepSeek-V4.1-Flash and GPT-5.6-Luna reach means of 78.1% and 77.9%, both above every trained policy. GPT-6.1-Sol leads them by 5.3 and 5.5 points, and a paired bootstrap over tasks keeps both gaps above zero (95% intervals 0.7 to 10.0 and 0.8 to 10.2). The two weaker models differ by only 0.2 points. All three use the same code, prompts, and protocol, suggesting that performance scales with model capability. Since the knowledge lives in the context and not in the weights, any foundation model with sufficient capability can serve as the base. Sec. 5.6 reports the cost of each model.

5.3 Why Does Success Hold under Randomization?

RoboScript rehearses only on clean scenes, yet retains over 95% of its success under randomization (Tab. 1 and Fig. 7). We verify this with per-task analysis and failure diagnosis.

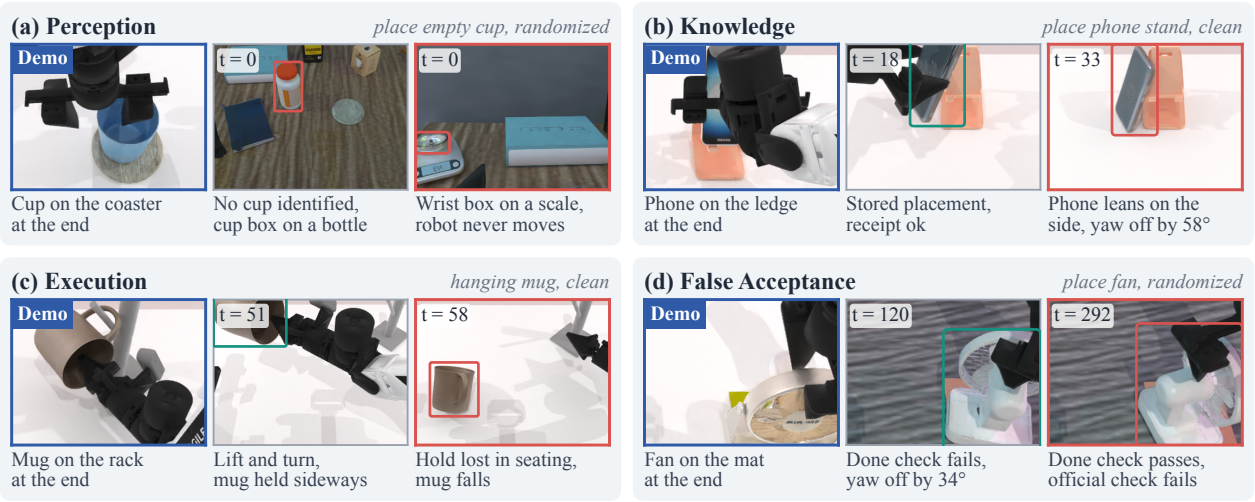


Figure 6 Failure cases. One failed evaluation episode of GPT-6.1-Sol for each cause and for false acceptance, with t the simulator step. Blue marks the demonstration and red the failure. (a) The cup is never found. (b) The stored placement leaves the phone leaning. (c) The mug drops while seated. (d) The agent's completion check accepts a fan that the official check rejects.

Per-task success is stable across settings. Fig. 5a plots clean versus randomized success for each task. For $\pi_{0.5}$, most tasks fall below the diagonal, and every task category loses 18.7 to 31.8 points under randomization. The three foundation models instead scatter on both sides of the diagonal. Averaged within each category, their success changes by at most 8.0 points between settings, and for GPT-6.1-Sol the change ranges from -7.0 to $+6.4$.

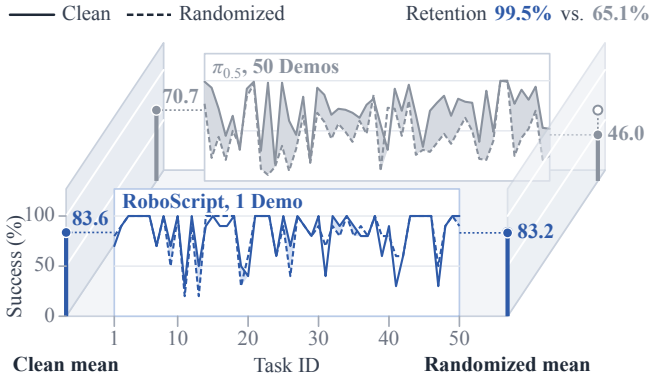


Figure 7 Performance retention under randomization. Per-task success of RoboScript (front) and $\pi_{0.5}$ (back), with the drop from clean to randomized shaded.

Knowledge failures do not increase under randomization. We label each failed GPT-6.1-Sol episode with a root cause (perception, knowledge, or execution) from its event log and check for false acceptances (Sec. D). Fig. 5b shows the distribution over 166 failed episodes (82 clean, 84 randomized). Knowledge failures account for 37 and 38, nearly unchanged between settings. Perception failures decrease slightly (22 to 17) and execution failures increase (23 to 29), consistent with randomized table heights making motions harder without affecting what the agent knows. False acceptance rates are also stable (28% and 29%). Fig. 6 shows one representative failure for each cause. The failures under randomization are physical, not epistemic, consistent with the semantic nature of the learned knowledge.

5.4 Where Do the Gains Come From?

Tab. 2 removes one component at a time. Deployment ablations use the frozen memory from the main run, the second group builds a memory without interacting with the world, and learning ablations relearn every memory from scratch. All groups use the same 10 tasks covering all five categories, with 10 episodes per task in each setting on shared seeds, so all comparisons are paired and we report exact McNemar p values.

The demonstration and the world are both necessary. The second group of Tab. 2 separates the two inputs of WICL. Writing the memory from the demonstration alone, without imitation or rehearsal in the world, gives 69.0%, against 78.5% when the same demonstration is followed by interaction with the world ($p = 0.003$). Without the world, the demonstration brings no benefit. Showing it to the agent at deployment

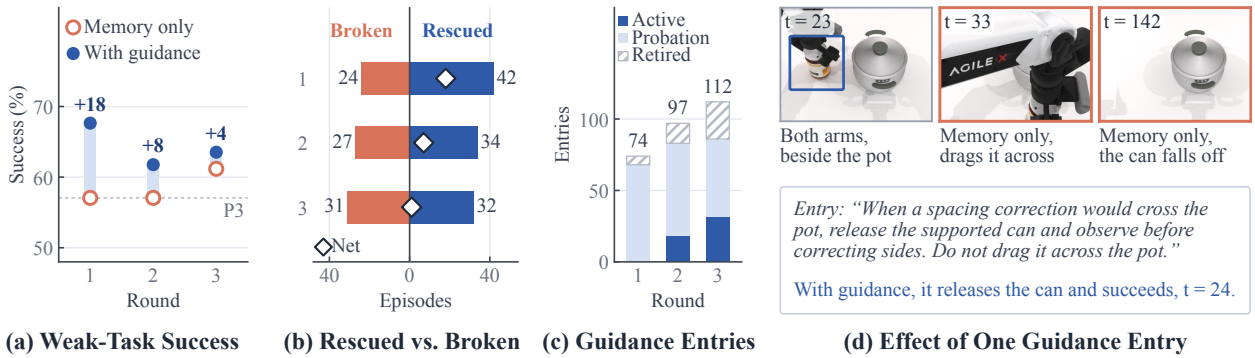
Figure 8 Deployment with a frozen Task Memory. Evaluation episodes of GPT-6.1-Sol, with t the simulator step. Blue marks the demonstration, red an error, and green success. (a) The agent mirrors the demonstration to the other arm. (b) A refused finish leads it to the skipped stage. (c) It grasps the dropped bottle again.

Table 2 Ablations. Success rate (%) of GPT-6.1-Sol on 10 tasks, with 10 episodes per task in each setting. Each variant is compared with the shaded full method above it on the same seeds. The second group builds the memory without interacting with the world, and the third group learns every memory from scratch.

Variant	Success Rate (%)		
	Clean	Rand.	Avg.
Deploying a frozen memory			
RoboScript	77.0	80.0	78.5
w/o arm check	78.0	72.0	75.0
w/o completion check	72.0	82.0	77.0
w/o recovery	68.0	70.0	69.0
Without the world			
Demonstration written to memory	69.0	69.0	69.0
Demonstration in deployment context	69.0	66.0	67.5
Instruction only	68.0	68.0	68.0
Learning			
RoboScript (relearned)	87.0	85.0	86.0
w/o demonstration	58.0	55.0	56.5
w/o privileged supervision	87.0	86.0	86.5

gives 67.5%, and a memory that holds only the instruction gives 68.0%, so the three variants differ by at most 1.5 points (pairwise $p \geq 0.76$). The world in turn needs the demonstration. Rehearsal from the instruction alone reaches 56.5%, which is 11.5 points below the same instruction-only memory deployed without rehearsal ($p = 0.004$).

The demonstration anchors learning. Without a demonstration, the agent begins rehearsal from an



(a) Weak-Task Success **(b) Rescued vs. Broken** **(c) Guidance Entries** **(d) Effect of One Guidance Entry**

Figure 9 Recursive self-improvement. GPT-6.1-Sol over three rounds on new clean seeds shared by both arms. (a) Success on the 17 weak tasks, with P3 dashed. (b) Episodes of all 50 tasks that guidance rescues or breaks. (c) Guidance entries by status. (d) An entry added in round 3 changes the outcome on one seed.

empty memory and the instruction alone, and the mean drops from 86.0% to 56.5%, the largest gap in the study. Rehearsal from trial outcomes alone still solves three tasks entirely (`turn_switch`, `click_alarmclock`, and `move_stapler_pad`), but two others fail to converge within the budget and score 0. The demonstration gives the agent a working starting point, without which convergence is unreliable. This variant also removes the demonstration-anchored completion check, so the drop reflects both losses.

Rehearsal corrects what the demonstration does not cover. Rehearsal tests the memory on new scenes before deployment and corrects the specifications that imitation leaves wrong, such as grasp and placement parameters. The 10 tasks of Tab. 2 exclude the tasks that are especially costly and hard to learn, and the effect of rehearsal is clearest on those tasks. On 11 such tasks, deploying the memory from imitation without rehearsal gives 46.1%, against 54.5% for the full method. The gain is 8.4 points on average and reaches 13.3 points under randomization ($p = 0.034$), where scenes differ most from the single demonstration. The rehearsal protocol applies strict edit budgets and convergence rules, so it changes the shared memory only where repeated trials show an error.

Anticipation and recovery at deployment. Fig. 8 illustrates the three deployment components that we ablate. Removing recovery, so that an episode ends at its first failed action, produces the largest drop in this group (78.5% to 69.0%). Removing the arm check, so that the agent always uses the arm from the demonstration instead of choosing per scene, reduces the mean to 75.0% (78 clean, 72 randomized). In the full method the agent switches arms in about half of successful grasps and achieves comparable success (93% and 95%), confirming that switching does not hurt. Removing the completion check, so that the agent no longer verifies the final state against the demonstration, gives 77.0% (clean -5, randomized $+2$). The clean loss comes from episodes that skip a manipulation stage.

Privileged supervision is not the source of performance. Learning has two privileged inputs. The demonstration stage observes ground-truth object state, and the reviewing role in rehearsal reads a ground-truth diagnosis of each failed attempt. We remove both at once and also restrict every role of the demonstration stage to deployable observations. Only the official success signal that each task records and the keypoints from the robot trajectory of the demonstration remain. This privilege-free variant reaches 86.5%, against 86.0% for the full method. Since the acting agent also uses only deployable observations during rehearsal and evaluation, the learned Task Memory does not depend on privileged information.

5.5 Recursive Self-Improvement

We evaluate the RSI loop (Eq. (3)) on the frozen memories of GPT-6.1-Sol over three rounds. Each round draws a new set of clean seeds with 10 episodes per task and evaluates the memory both with and without guidance on the same seeds, so the paired gap isolates the effect of reflection. The reflection agent reads only deployable observations and whether each episode succeeded.

On the 17 weak tasks (clean success $\leq$8 of 10 before reflection), guidance rescues more episodes than it breaks

Table 3 Foundation models and cost. DS is DeepSeek, and Avg. is the success rate (%) of Tab. 1. Exits counts the tasks that pass the S1 exit rule, and Instances is the mean number of S2 instances per task. Evaluation columns are means per episode, with tokens in millions. Cost covers learning and evaluation at list API price.

Model	Avg.	S1		S2	Evaluation episode			Cost
		Exits	Attempts	Instances	Calls	Tokens	Steps	(USD)
DS-V4.1-Flash	78.1	50/50	301	11.1	53.1	3.89	183	739
GPT-5.6-Luna	77.9	47/50	518	14.5	62.2	2.61	190	308
GPT-6.1-Sol	83.4	49/50	329	12.5	54.7	2.33	149	1,911

in every round, with net gains of +18, +8, and +4 episodes (Fig. 9a). On all 50 tasks the net gains are +18, +7, and +1 (Fig. 9b). The gains concentrate on harder tasks where the reflection agent identifies failure patterns that the original memory did not address. The number of guidance entries grows from 74 to 97 to 112 (Fig. 9c). Fig. 9d shows one episode in which a round 3 entry changes the action the agent takes. This confirms that frozen weights do not prevent improvement when the knowledge is stored in readable context.

5.6 Cost of Learning and Deployment

Tab. 3 reports the success of each foundation model together with the cost of learning and deployment.

Learning and deployment. The demonstration stage takes 301 to 518 attempts across the three models, and rehearsal averages 11.1 to 14.5 instances per task. GPT-6.1-Sol passes the demonstration exit rule on 49 of 50 tasks, DeepSeek-V4.1-Flash on all 50, and GPT-5.6-Luna on 47. Tasks that do not pass are frozen with the best available memory (Sec. D). At deployment, an evaluation episode with GPT-6.1-Sol takes on average 54.7 model calls, uses 2.33 million tokens, and runs for 149 simulation steps.

Cost. At list API prices, learning all 50 tasks and evaluating 1,000 episodes costs 308 USD with GPT-5.6-Luna and 1,911 USD with GPT-6.1-Sol. Evaluation accounts for the majority of the cost (Fig. 13). DeepSeek-V4.1-Flash generates about 19 times more output tokens per call than GPT-6.1-Sol, so despite a lower per-token price its total reaches 739 USD. Once frozen, the Task Memory can be reused across episodes without additional learning cost.

5.7 Real-Robot Deployment

We deploy RoboScript on a physical dual-arm PiPER robot (Fig. 10) to test whether Task Memories learned in simulation transfer to a different embodiment. The PiPER has two 6-DoF arms with parallel-jaw grippers and a different kinematic structure than simulated ALOHA used during rehearsal.

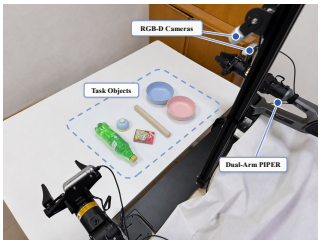


Figure 10 Real-robot setup with a dual-arm PiPER and RGB-D cameras.

Setup. We take the frozen Task Memories from the main experiment (GPT-6.1-Sol) and apply a lightweight adapter that remaps the embodiment-specific parameters for the PiPER robot. The adapter maps coordinate conventions, arm kinematics, and gripper parameters while leaving the task workflow, object roles, and relative specifications unchanged. The original demonstrations and rehearsal are not repeated. We plan the full action sequence from the memory offline so the robot executes each step without online replanning, and test four tasks with 10 episodes each.

Results. Fig. 11 shows execution frames for the four tasks. RoboScript succeeds on 10 of 10, 7 of 10, 7 of 10, and 6 of 10 episodes on Click Bell, Grab Roller, Move Playingcard Away, and Stack Bowls Two, respectively. These tasks cover pressing, bimanual grasping, pick-and-place, and stacking. Failures are mainly caused by gripper misalignment and depth estimation errors on real sensors. The memories were rehearsed entirely in simulation on a different robot, yet they produce working behavior on real hardware through parameter remapping alone. This validates WICL's prediction that semantic knowledge transfers across worlds.

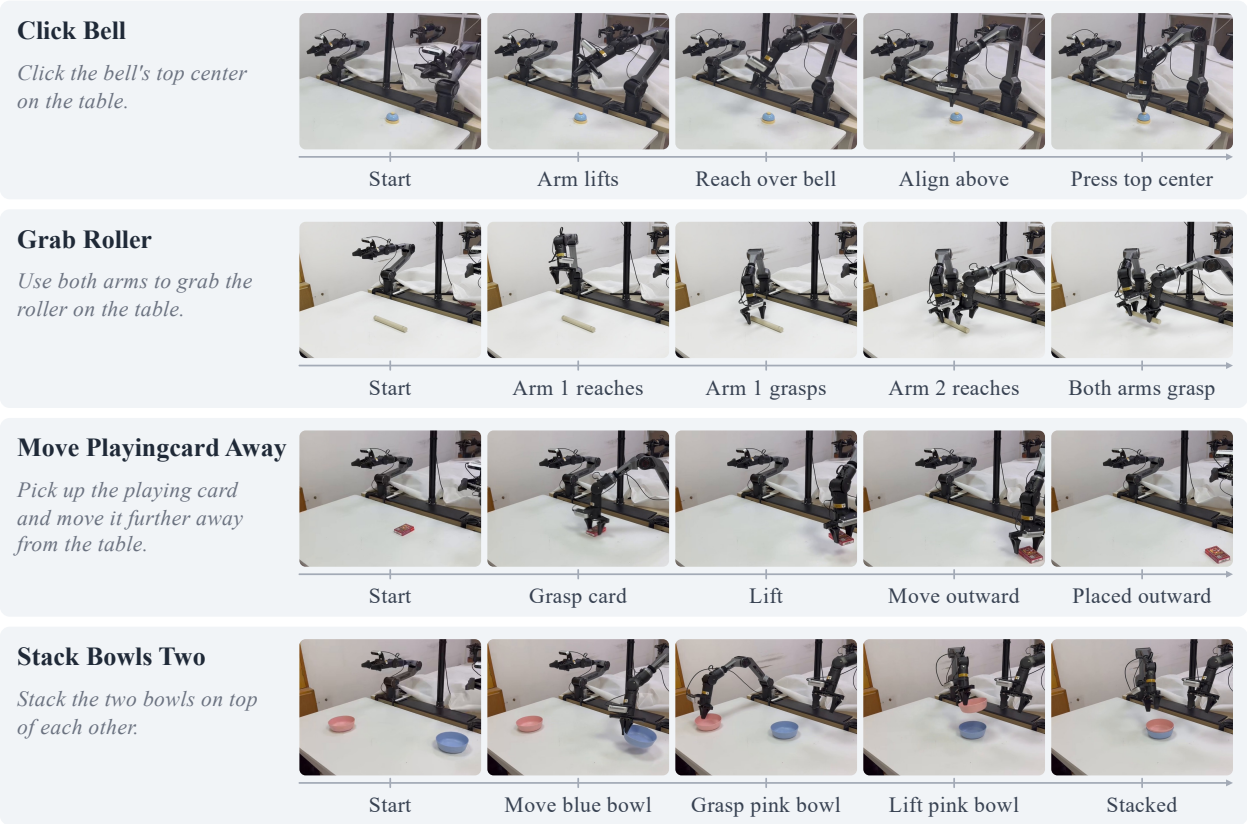


Figure 11 Execution frames for four real-robot tasks. Each row shows key frames from a successful episode. The Task Memory was rehearsed on a simulated ALOHA and transferred to the PiPER through a lightweight adapter without re-rehearsing.

6 Conclusion

We have introduced World In-Context Learning (WICL), a paradigm that extends in-context learning with feedback from a world, and implemented it in RoboScript, a modular infrastructure for agentic manipulation with strict data isolation. On all 50 tasks of RoboTwin 2.0, the agent builds its own skills, learns each task from one demonstration and rehearsal in a simulator, and reaches 83.4% mean success across clean and randomized settings without gradient updates, exceeding every trained action policy. The learned knowledge is semantic, so it transfers across visual distributions, works with three different foundation models, and improves through reflection over successive rounds. A lightweight adapter transfers the same memories to a physical PiPER robot with a different kinematic structure, producing working behavior without re-recording any demonstration or repeating any rehearsal. These findings suggest that task skill need not reside in network weights, and that with a world in the loop, a robot can learn a manipulation task in its context as well as it learns one in its weights.

Appendix

We first extend the discussion of WICL and its relation to other paradigms (Sec. A), then describe the system architecture (Sec. B) and the method details including skills, perception, learning, and data isolation (Sec. C), present additional quantitative (Sec. D) and qualitative (Sec. E) results, and finally discuss limitations and future work (Sec. F).

Sec. A Extended Discussion about WICL

Sec. B System Architecture

Sec. C Method Details

Sec. D Additional Quantitative Results

Sec. E Additional Qualitative Results

Sec. F Limitations and Future Work

A Extended Discussion about WICL

We discuss how WICL relates to established learning paradigms, what distinguishes it from prior work, and why it has become feasible for physical manipulation.

Relation to Other Learning Paradigms. Skill acquisition differs along two dimensions. The first is the source of learning, either demonstrations alone or feedback from a world. The second is where the learned skill is stored, either in network weights or in the context of a frozen model. Imitation learning writes demonstrations into weights. Reinforcement learning adds a world whose feedback is also written into weights. In-context learning stores demonstrations in context. WICL adds a world whose feedback is also stored in context (Fig. 1a). The world provides a feedback signal that demonstrations alone cannot offer, because a demonstration is recorded in a different scene and cannot tell the learner what will happen in the scene in front of it. Writing this feedback into context instead of weights avoids the distribution coupling that limits weight-based methods.

Distinction from In-Context RL and Experiential Learning. In-context reinforcement learning [43, 44] also improves within the context, but its weights are first trained on learning histories from the target domain, so the model itself is specialized. WICL uses a general foundation model without such training. Experiential learning in LLM agents [66, 74, 94] is the closest relative. Some of these methods improve through repeated trials on the same instance [66], which conflates memorization with generalization. The split between $\mathcal{E}_{\text{reh}}$ and $\mathcal{E}_{\text{test}}$ in WICL rules this out and ensures that every reported success measures generalization.

WICL in Physical Manipulation. Applying WICL to physical manipulation introduces requirements that do not arise in software or text domains. The context must be grounded in camera observations of a scene that changes as the robot acts. The feedback from the world includes physical outcomes such as whether an object was grasped, not only textual signals. The context must therefore record task-relevant semantics from visual observations while remaining independent of the visual appearance of any particular world. In our implementation, a linter enforces this by rejecting coordinates, ground-truth terms, and instance identifiers.

Feasibility Conditions. Simulators and robot APIs have been available for years [53], but WICL also requires a model that can use tools over a long context, read feedback, and accumulate experience. The progress of coding agents on real software repositories shows that foundation models have crossed this threshold [76, 83]. Their terminal is their world, and a simulator can now serve the same role for a robot. The remaining requirement is a world that is faithful to the task, resettable, and keeps rehearsal apart from evaluation instances.

B System Architecture

Two planes. As shown in Fig. 12, RoboScript separates a control plane from an agent plane. The control plane runs the protocols of the lifecycle. It opens and closes episodes, selects instances, judges success, and

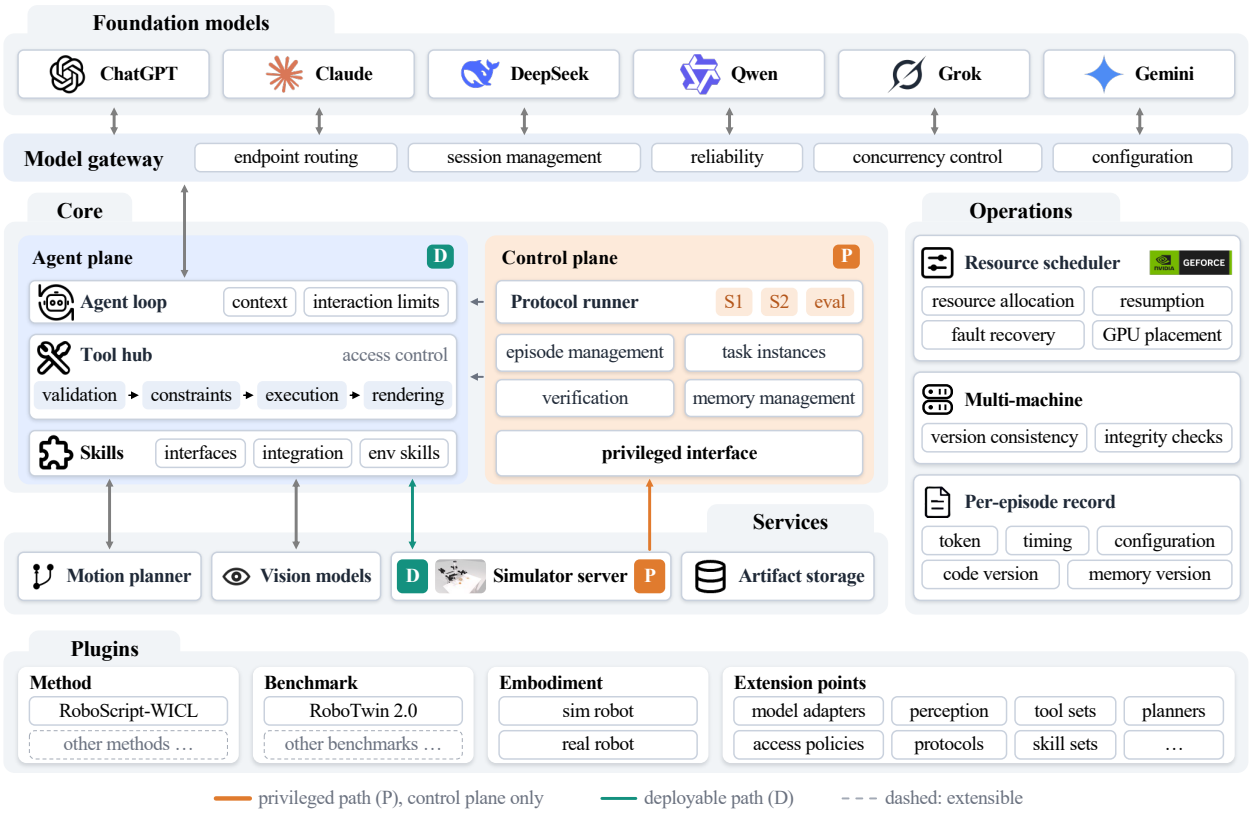


Figure 12 Detailed architecture of RoboScript, expanding Fig. 2. The model gateway handles endpoint routing, session management, and reliability. The core separates an agent plane (agent loop, tool hub, skills) from a control plane (protocol runner, memory management, verification). Services, operations, and plugins are independently replaceable.

stores and freezes memories. It is the only part of the system that can reach privileged simulator state. The agent plane contains the foundation model, the tool hub, and the tools. The simulator, the motion planner, and the vision models run as separate services. Every message between processes is a typed and versioned contract, and large arrays are passed by reference. Each component can therefore be developed, tested, and replaced on its own.

Model gateway. Every model call, whether from the acting agent, a reviewing role, or perception, passes through one gateway. The gateway exposes a single interface over several provider endpoints. It keeps a session on the endpoint that served it, moves to another endpoint when one fails, and adapts the number of concurrent requests to what each endpoint sustains. Changing the foundation model of a run is a change of configuration, and the same setting applies to every role and to perception.

Tool hub. The hub sits between the model and the tools and applies the rules of each stage. When an episode opens, the hub fixes the tools that each role may see from the stage, the role, and the data tier. Every call then passes through one ordered pipeline that validates it, checks the budgets and the stage rules, executes it, and renders the result. Evaluation stages admit tools of the deployable tier only, and the hub refuses any privileged call made in them, including a call that one tool makes on behalf of another.

Scheduling and reproducibility. For each task, a job graph runs imitation, rehearsal, and deployment in order. A scheduler places jobs on GPUs according to their measured memory, starts the services of a GPU when a job first needs them, and resumes interrupted jobs from their records. Infrastructure failures, such as a lost connection to a model endpoint, are retried and are not counted as task failures. Across machines, tasks are assigned to workers that run the same code version and configuration, and their results are copied back and verified with checksums. A run does not start from an unversioned or modified code tree. Every

episode records its code version, configuration, memory version, token usage, and duration, and the event log of an episode is sufficient to reconstruct everything the model saw.

Extensibility. All major components, including benchmarks, embodiments, perception backends, motion planners, model clients, and methods, are registered through plugin interfaces with conformance tests. The method in this paper is one such plugin built on these mechanisms.

C Method Details

C.1 Skills and Specifications

Shared and environment-specific skills. Skills are divided by whether they depend on the environment. Shared entry points cover common operations such as observing, grasping, placing, corrective motions, reachability checks, arm selection, and coordinated bimanual motion. These entry points hold no convention of any environment. Environment conventions, including coordinate frames, end-effector-to-fingertip offsets, gripper normalization, the collision world, and arm names, enter the shared entry points as data through a binding. An operation that holds only in a particular environment is an environment-specific skill, and it becomes visible only when that environment is loaded.

Specifications instead of poses. Grasps and placements are described by specifications instead of poses. A grasp specification names the part to hold and the approach relative to that part. A placement specification names the target and the offset and orientation relative to it. The frames of these specifications are perceived objects, their parts, landmarks, or the robot base, and no specification stores the absolute position of an object. At execution time, the skill solves the specification on the perceived geometry of the current scene, mirrors it when the other arm acts, and keeps the first solution that the motion planner can reach. The parameters of the specifications belong to the Task Memory and are learned during rehearsal.

Receipts. Every physical action returns a receipt. The receipt states whether the action succeeded, the pose that was actually reached, and whether an object is held. It also reports the deviation from the target and a reason when the action failed. The model therefore learns from what happened, not from what it commanded.

C.2 Skill Building

Coverage of the demonstrations. Skill building starts from the demonstrations of all tasks. Each demonstration is cut at its gripper events into segments of seven kinds, namely grasp, place, carry, touch, release, move, and return home. A segment is explained when an existing entry point or written skill, with parameters fitted to the demonstration, reproduces it from the state where the segment starts. Each task is checked in two ways. Every segment is replayed from its own start, and the plan of the whole task is also rolled out once from the reset, so that each call starts where the previous calls left the scene. Segments that no entry explains are gaps, and gaps of the same kind and failure class are grouped into clusters.

Written skills. Each cluster is handed to a coding agent that works in a sandbox with fixed budgets of case runs, wall-clock time, and tokens. A written skill has two parts. The action part runs at the deployable tier, exactly as the agent runs it in rehearsal and deployment, and it uses only the action API, the object handles it receives, and its parameters. The fitting part runs only during skill building and may read the recording and the ground-truth scene. It returns the parameters that reproduce one segment. These parameters hold no absolute position, since the fit is repeated on the demonstration moved across the table and a parameter that moves with it by more than 1 cm rejects the fit.

Gate. The driver accepts a skill only through its own checks and never relies on the report of the coding agent. A static check rejects imports outside an allowed set, task names, object labels, and position literals. The skill must then explain the gaps it was written for. A transfer check runs its fitted parameters on the same segment of up to three other demonstrations whose object lies at least 5 cm away, and rejects a skill that explains none of them, so that a replay cannot pass as a skill. Finally, all sweeps of the round run again with the accepted skills and the candidates, and a candidate is dropped when its confirmed losses outweigh its gains. Every process that imports a written skill runs in the sandbox.

Table 4 Hyperparameters. Values used in all runs. k counts the rehearsal instances of a task.

Stage	Parameter	Value
S1	Exit rule	3 of the 5 most recent attempts without hints pass
S1	Hints	First attempt, then one after every 2 consecutive failures without hints
S1	Attempt cap	60, including attempts with hints
S2	Counted-instance cap	60 (20 in the ablations)
S2	Tries per instance	3, only the first counts
S2	Early stop	3 first-try successes among the 5 most recent counted instances
S2	Step size at instance k	$\alpha_k = \max(0.2,\ 10/(9+k))$
S2	Edit budget at instance k	$\max(1,\ \mathrm{round}(4\alpha_k))$
All	Reasoning effort	High for every role

Stopping and freezing. Rounds repeat until coverage stops growing, measured by explained segments, task rollouts that reach the end, official successes of the rolled-out plans, and completed stages. The accepted skills are then frozen as the skill set of the environment. The task plans that the builder uses to measure coverage stay in its records and never enter the skill set, a Task Memory, or any input of learning or deployment.

C.3 Perception

Perception separates geometry from semantics. Positions, sizes, and orientations come from the depth images and the calibrated cameras, and foundation models are asked only which object or part is which. At the start of an episode, the agent identifies the objects of the task once, and an identity is accepted only when a majority of independent detectors agree. An ambiguous result is reported to the agent instead of resolved silently. During the episode, objects are tracked through the grasp relations that the actions record instead of being detected again, which keeps their identities stable while the arms occlude them. Points that belong to the robot are removed with its kinematic model.

C.4 Task Learning

Imitation. S1 involves three roles. A demonstration reader studies the recorded demonstration, including its frames, the motion of the robot, and the key operations extracted from gripper events. It writes the first version of the Task Memory. An acting role attempts the task on the instance of the demonstration. Some attempts may consult the demonstration as hints, and the others rely on the memory alone. A reviewing role reads the record of each attempt and revises the memory. An attempt passes when the official success check fires and the key operations of the demonstration were carried out in order. S1 ends once the attempts without hints pass consistently within a window of recent attempts. Tab. 4 lists the values used in all runs.

Rehearsal. S2 runs on new training instances in order, and only the first attempt on each instance counts. If it fails, the reviewing role edits a cached copy of the memory, and the instance is retried with that copy. The edits enter the shared memory only if a retry succeeds. The number of edits allowed per revision and the step size of numerical updates both decrease as rehearsal proceeds, so the memory changes quickly at first and stabilizes later. Rehearsal stops when enough recent first attempts succeed within a sliding window, or after a fixed instance budget. If the budget is reached before the early-stopping rule triggers, the version with the best record of first attempts is frozen.

Task Memory. The Task Memory is a structured Markdown document. Its sections hold the instruction, the roles of the objects and how to find them, the workflow of each stage with its completion criteria, lessons, and notes. A separate table holds typed parameters such as grasp and placement specifications. Reviewing roles change the memory only through typed edits, each with a stated reason, and every version is stored and hashed. Before a version is committed, a linter rejects simulator names, evaluator terms, instance identifiers, absolute positions, distances measured from the robot, and fixed assignments of arms. The memory thus stays semantic and independent of any instance.

Table 5 Data available to each role. D denotes the deployable tier and P the privileged tier. Every role that acts on a new instance uses the deployable tier only.

Stage	Role	Tier	Demonstration	Privileged data	Edits memory
S1	Demonstration reader	P	full	tools and demonstration state	yes
S1	Acting agent	P	as hints only	tools	no
S1	Reviewing role	P	full	diagnosis and demonstration state	yes
S2	Acting agent	D	final state	none	no
S2	Reviewing role	D + diagnosis	none	diagnosis	cached copy
P3	Acting agent	D	final state	none	no

C.5 Completion Check

Before the agent may declare a task done, it checks the scene against the final state of the demonstration. The check uses the completion criteria in the memory and a visual comparison with the final frames of the demonstration. Only the success check of the simulator credits an episode.

C.6 Information Boundary

Two tiers. Tab. 5 lists the data available to each role. The deployable tier contains RGB-D images from the head and wrist cameras, camera intrinsics and extrinsics, measured and commanded joint states, gripper openings, feedback from the motion planner, the step count, and the instruction. The privileged tier contains ground-truth object poses, asset annotations and meshes, articulation states, contacts, ground-truth segmentation, the success check, and expert trajectories.

Enforcement. The boundary is enforced by construction. Agent-side code reaches the simulator only through an interface that returns deployable data, and only the control plane holds the privileged interface. The hub derives the tools of each role from its tier and refuses privileged calls in evaluation stages. Deployment mounts no privileged tool at all, and observations, error messages, and tool outputs are filtered so that no privileged value passes through them. Automated tests check that every evaluation stage refuses privileged tools, that a single module alone may access privileged state, and that no text shown to the model in rehearsal or deployment is tied to a particular task.

Supervision during learning. Privileged data serves only as supervision during learning. Every role in imitation runs at the privileged tier, since imitation takes place on the instance of the demonstration. In rehearsal, the reviewing role receives a ground-truth diagnosis of each failure, stated in terms of the objects and roles of the episode. This supervision reaches the acting agent only through the memory, which must pass the linter. A privilege-free variant that removes all of this supervision is evaluated in Sec. 5.4.

D Additional Quantitative Results

Statistical uncertainty. Over the 1,000 deployment episodes of each model, the 95% Wilson interval of the mean is 81.0 to 85.6 for GPT-6.1-Sol, 75.2 to 80.4 for GPT-5.6-Luna, and 75.4 to 80.6 for DeepSeek-V4.1-Flash. Resampling tasks instead of episodes gives 77.5 to 89.0, 70.7 to 84.6, and 71.4 to 84.4, which reflects the spread of difficulty across tasks. The frozen GPT-6.1-Sol memories, evaluated on four further disjoint seed blocks in the clean setting, reach 83.4%, 84.6%, 85.2%, and 83.0%.

S1 exit. DeepSeek-V4.1-Flash passes the S1 exit rule on all 50 tasks. GPT-6.1-Sol passes on 49 (`place_fan` does not exit). GPT-5.6-Luna passes on 47 (`place_fan`, `put_object_cabinet`, and `place_cans_plasticbox` do not exit). A task that does not exit is frozen with the best available memory, defined as the memory version that has at least one official success and the highest Wilson 95% lower bound on first-try success. In this run, every non-exiting task has at least one such version, so none scores 0 by default.

Ablation details. Tab. 2 uses 10 tasks that cover the five categories of Tab. 8. The tasks are `adjust_bottle`, `click_alarmclock`, `handover_block`, `move_stapler_pad`, `open_laptop`, `pick_diverse_bottles`, `place_-`

Table 6 Failure attribution. Failed evaluation episodes by physical root cause, attributed from the full event log and action receipts. False acceptance counts episodes whose own completion check accepted a state that the official check rejects, and cuts across the three causes. Luna's clean execution count includes 2 tool timeouts.

	GPT-5.6-Luna		GPT-6.1-Sol	
Cause	Clean	Rand.	Clean	Rand.
Perception	9	13	22	17
Knowledge	28	20	37	38
Execution	64	87	23	29
Total	101	120	82	84
False acceptance	17 (17%)	15 (13%)	23 (28%)	24 (29%)

`can_basket`, `place_container_plate`, `stack_blocks_three`, and `turn_switch`. All groups use evaluation seeds that are disjoint from all learning and rehearsal seeds. Without the world, the first variant writes the memory from the same demonstration package as imitation but makes no call to the simulator. The second deploys a memory that holds only the instruction and shows the deploying agent the demonstration frames and robot keypoints, without its ground-truth state. The third deploys the instruction-only memory alone. The privilege-free variant runs every role of imitation at the deployable tier, reads no ground-truth state of the demonstration, and gives the reviewing role no diagnosis. The learning group caps rehearsal at 20 counted instances per task. The 11 harder tasks used to evaluate rehearsal are `hanging_mug`, `move_can_pot`, `open_microwave`, `pick_dual_bottles`, `place_bread_skillet`, `place_dual_shoes`, `place_fan`, `put_bottles_dustbin`, `put_object_cabinet`, `rotate_qrcode`, and `stack_bowls_three`. Fig. 14 shows the paired changes per task.

Unseen object instances. In 39 tasks the objects vary across instances, and 78% of the evaluation episodes of these tasks use object instances different from the demonstration. Within each task, these episodes show no significant difference in success from episodes with the demonstrated instances (Cochran–Mantel–Haenszel p of 0.97, 0.50, and 0.97 for GPT-6.1-Sol, GPT-5.6-Luna, and DeepSeek-V4.1-Flash). In eight tasks, no evaluation episode uses the demonstrated instances, and success still ranges from 75% to 100%.

Failure attribution. Each failed evaluation episode is labeled with the earliest unrecovered root cause that made official success unreachable. We use three categories. Perception failures misidentify an object or mislocate a part. Knowledge failures arise when the agent executes the workflow correctly but the stored specification does not match the scene or the instruction. Execution failures include unreachable targets, empty grasps, dropped objects, and running out of steps during recovery. The reviewer also records false acceptances, where the agent's completion check accepts a state that the official check rejects. Tab. 6 reports the full attribution for GPT-6.1-Sol and GPT-5.6-Luna. To check that the labels do not depend on the labeling model, a different model independently relabels 30 failed episodes. The two agree on the root cause in 26 of 30 episodes (Cohen's $\kappa = 0.82$), and on another 30 failed episodes the distribution of causes from the second model matches that of the first.

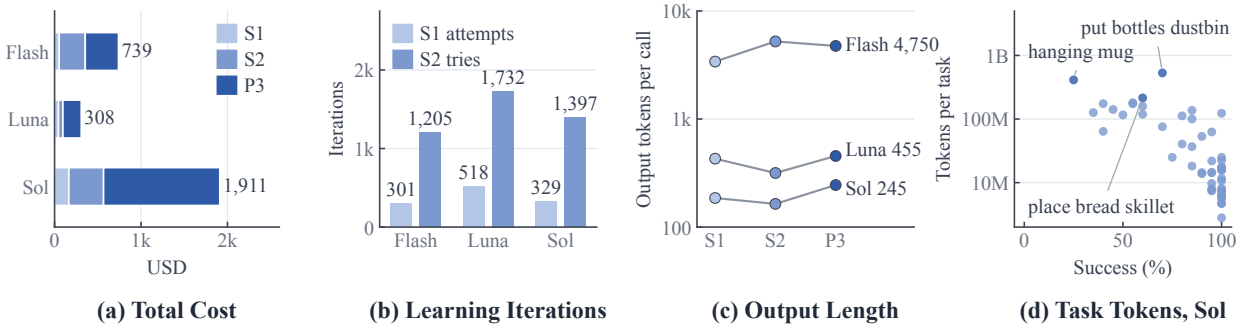
Cost breakdown. Tab. 7 reports, for each foundation model and stage, the number of episodes, the mean number of model calls, and the mean numbers of input and output tokens. Attempts redone after infrastructure failures are excluded in every stage. A deployment episode takes a median of 6.4 minutes with GPT-6.1-Sol, 10.4 with GPT-5.6-Luna, and 3.6 with DeepSeek-V4.1-Flash, and about 80% of this time is spent waiting for the foundation model. The simulator, the motion planner, and the vision models run on 16 NVIDIA RTX 4090 GPUs across 2 servers, and the foundation models are accessed through their provider APIs.

E Additional Qualitative Results

Task Memory structure. Fig. 15 shows the frozen Task Memory of `place_object_basket` as the actor receives it at evaluation. The memory is organized into five sections. The instruction and object roles describe

Table 7 Cost per episode. Means over the acting agent's episodes in each stage, where S1 and S2 episodes are imitation and rehearsal attempts. Input tokens include cached input, with its share in parentheses. USD uses each model's standard list API price in October 2026, and the off-peak rate of DeepSeek-V4.1-Flash is half of it. Reading the demonstration and reviewing attempts are excluded. Attempts redone after infrastructure failures are excluded.

Model	Stage	Episodes	Model calls	Input tokens (M)	Output tokens (k)	USD
GPT-6.1-Sol	S1	329	22.6	0.73 (84%)	4.2	0.34
	S2	1,397	18.4	0.52 (86%)	3.0	0.22
	P3	1,000	54.7	2.32 (78%)	13.4	1.33
GPT-5.6-Luna	S1	518	27.4	0.88 (80%)	11.7	0.06
	S2	1,732	12.0	0.33 (82%)	3.8	0.02
	P3	1,000	62.2	2.58 (73%)	28.3	0.21
DeepSeek-V4.1-Flash	S1	301	18.2	0.94 (93%)	61.7	0.10
	S2	1,205	17.0	0.94 (89%)	88.7	0.14
	P3	1,000	53.1	3.64 (94%)	252.2	0.39



(a) Total Cost **(b) Learning Iterations** **(c) Output Length** **(d) Task Tokens, Sol**

Figure 13 What the cost consists of. (a) Cost at list API price by stage, mostly from evaluation. (b) Total S1 attempts and S2 tries. (c) Output tokens per call of the actor. DeepSeek-V4.1-Flash writes about 19 times more than GPT-6.1-Sol in evaluation. (d) Per-task tokens and success of GPT-6.1-Sol. Less successful tasks take more tokens.

the task and how to identify each object. The workflow lists eight stages in order, each with its goal, the arm to use, the API calls, and the condition that marks it as done. The parameter table holds 21 typed specifications, including grasp poses, placement offsets, and scalar thresholds. Lessons record failure patterns that the reviewing role identified during rehearsal. In this example, rehearsal added stage 3 (a handover to the other arm) and both lessons, shown in blue. The text shown is quoted from the stored file and represents the full context the acting agent receives at every step during evaluation.

Context that changes the outcome. Fig. 16 shows two cases where a single piece of learned context changes the evaluation outcome on the same seeds.

In (a), rehearsal adds one lesson to the Task Memory of `move_stapler_pad`. Without this lesson, the agent finishes on xy overlap alone and the official check rejects the episode. With the lesson, the agent verifies containment before finishing. On the same clean seeds, success rises from 4 of 10 to 9 of 10.

In (b), the reflection loop writes two guidance entries for `handover_block` over three rounds. The first entry (added in round 1) corrects a misinterpretation of abbreviated wording that causes the agent to stop after the donor hold. The second entry (added in round 3) tells the agent to verify placement against the visible pad edges and recenter if displaced. With both entries, success on the same seeds rises from 4 of 10 to 10 of 10. These examples illustrate that the WICL loop writes targeted, readable corrections into the context and that small semantic changes can recover multiple episodes.

Table 8 Per-task success rate (%) on RoboTwin 2.0. Tasks are grouped by the five categories of Sec. 5.1, with the number of tasks in parentheses. RoboScript uses 10 evaluation episodes per task and setting, and $\pi_{0.5}$ is taken from the official leaderboard with 100 episodes. C and R denote the clean and randomized settings, and each category ends with its mean.

Task	$\pi_{0.5}$		RoboScript (Flash)		RoboScript (Luna)		RoboScript (Sol)	
	C	R	C	R	C	R	C	R
Articulated objects (4)								
`open_laptop`	93	68	60	60	90	100	90	100
`open_microwave`	86	59	100	100	100	100	100	100
`put_object_cabinet`	39	22	60	40	30	10	90	80
`turn_switch`	52	25	100	100	90	90	100	90
Mean	*67.5*	*43.5*	*80.0*	*75.0*	*77.5*	*75.0*	*95.0*	*92.5*
Oriented parts (11)								
`adjust_bottle`	99	77	80	90	70	70	70	80
`beat_block_hammer`	93	23	90	80	90	80	90	90
`hanging_mug`	17	14	0	0	30	0	30	20
`move_stapler_pad`	20	18	100	90	70	90	90	100
`place_dual_shoes`	70	45	60	80	70	60	80	80
`place_fan`	66	26	10	20	30	30	40	70
`place_mouse_pad`	34	31	100	100	100	100	100	90
`place_phone_stand`	65	37	90	100	70	80	80	90
`place_shoe`	82	50	90	80	90	90	80	80
`rotate_qrcode`	90	21	70	30	70	60	30	60
`scan_object`	49	40	0	30	30	20	60	60
Mean	*62.3*	*34.7*	*62.7*	*63.6*	*65.5*	*61.8*	*68.2*	*74.5*
Functional points on a receiver (10)								
`place_bread_basket`	63	60	90	80	80	70	100	100
`place_bread_skillet`	76	47	30	50	20	20	60	60
`place_burger_fries`	82	86	90	80	100	100	100	90
`place_can_basket`	56	10	60	80	70	70	70	40
`place_cans_plasticbox`	31	73	90	80	100	80	100	100
`place_container_plate`	92	72	90	90	100	60	90	90
`place_empty_cup`	96	80	90	80	100	90	100	90
`place_object_basket`	70	29	80	90	60	80	90	80
`place_object_scale`	78	38	80	90	100	90	100	100
`place_object_stand`	85	47	90	90	80	100	90	80
Mean	*72.9*	*54.2*	*79.0*	*81.0*	*81.0*	*76.0*	*90.0*	*83.0*
Relative placement (10)								
`handover_block`	30	12	70	60	90	80	70	50
`move_can_pot`	60	10	80	60	50	40	50	20
`move_pillbottle_pad`	46	32	90	90	100	80	90	100
`place_a2b_left`	71	49	100	90	100	90	100	100
`place_a2b_right`	68	39	100	100	100	90	100	100
`stack_blocks_three`	77	25	60	80	100	100	100	100
`stack_blocks_two`	91	43	100	100	100	100	100	100
`stack_bowls_three`	81	52	40	70	20	20	30	50
`stack_bowls_two`	94	70	90	70	90	80	90	90
`stamp_seal`	53	21	70	90	90	80	100	100
Mean	*67.1*	*35.3*	*80.0*	*81.0*	*84.0*	*76.0*	*83.0*	*81.0*
Single objects (15)								
`blocks_ranking_rgb`	72	44	80	70	90	80	100	100
`blocks_ranking_size`	45	21	70	100	100	100	100	100
`click_alarmclock`	65	50	100	90	90	90	100	100
`click_bell`	31	35	100	100	100	100	100	100
`dump_bin_bigbin`	92	84	70	60	70	60	70	70
`grab_roller`	99	97	100	100	90	100	100	100
`handover_mic`	98	6	90	100	100	100	100	100
`lift_pot`	98	35	100	90	100	100	100	90
`move_playingcard_away`	84	65	90	100	100	100	100	100
`pick_diverse_bottles`	66	42	30	70	30	50	50	30
`pick_dual_bottles`	72	57	90	100	100	100	40	60
`press_stapler`	79	63	100	100	90	90	100	100
`put_bottles_dustbin`	78	50	70	30	50	40	60	80
`shake_bottle`	100	100	100	90	100	100	100	100
`shake_bottle_horizontally`	99	100	100	100	100	90	100	100
Mean	*78.5*	*56.6*	*86.0*	*86.7*	*87.3*	*86.7*	*88.0*	*88.7*
Average	70.7	46.0	77.8	78.4	79.8	76.0	83.6	83.2

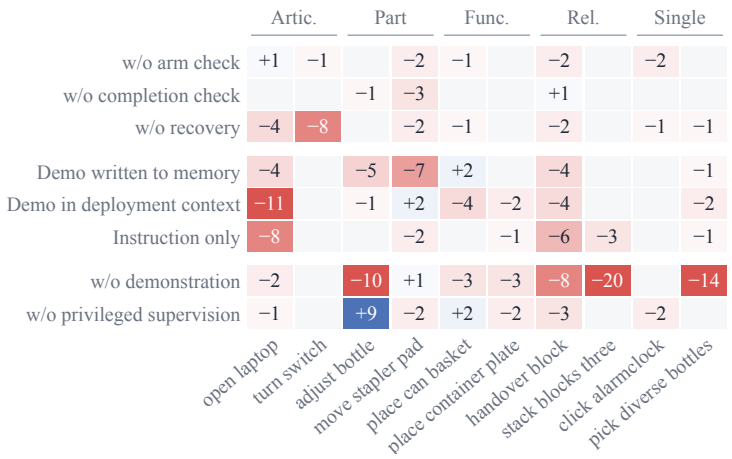


Figure 14 Paired changes per task in the ablations. Each cell is the change in successes over 20 episodes on the same seeds, with blue a gain and red a loss. Each variant is compared with the control of its group in Tab. 2, where variants without the world are compared with the frozen memory of the main run. Tasks that do not converge in rehearsal without the demonstration count as failures.

INSTRUCTION from the demonstration	Grab the bright sleek toycar, drop it in the basket with perforated sides, then move the basket with perforated sides.
OBJECTS role, description, x axis	`toycar`: the object to drop into the container; a small rigid toy vehicle, grasped across its body [...]. x toward the front nose. `basket`: receives the object and is then moved while containing it [...]. x toward the raised handle crossbar end.

WORKFLOW stages with goal, arm, calls, done	`1 prepare → 2 grasp_toycar → 3 handover_toycar → 4 lift_toycar → 5 drop_toycar → 6 retreat → 7 grasp_basket → 8 move_basket`
	`5 drop_toycar`: Move above the basket opening with the recorded tilt and release the toycar so it falls into the basket. [...]
	`call: place_actor(arm=<arm>, obj=toycar, place={toycar_drop}, pre_dis={drop_pre_dis}, dis={drop_dis}, is_open=true, constrain='align'); [...]`
	`done: gripper_open(<arm>, {open_fraction}) and not holding(<arm>)`

PARAMETERS fitted from the demonstration	`toycar_drop = place_spec(target=basket top_center, frame=target, offset_m=[0.0781, 0.0067, 0.0234], orient=relative, rpy_deg=[-42.7, 84.1625, -138.2], functional=bottom, [...])` `lift_height = 0.1 m` [...] 19 more grasp, path, place, and done specifications
LESSONS written from failures	`L2` *When* the object has been released with the fitted tilted placement and the empty gripper must clear the basket, *do* keep the arm selected for the object through retreat and use `back_to_origin(arm=<arm>)` for the empty gripper, allowing a planned path and orientation change. Do not replace this with a long straight world displacement at the release orientation, even if its endpoint appears clear. [...] `L1` [...]
NOTES free-text cautions	The demonstration ends with the loaded basket still grasped, not set down. [...]

Figure 15 Task Memory. The frozen Task Memory of `place_object_basket` learned by GPT-6.1-Sol, as the actor receives it at evaluation. The left column names each section and what it holds. Rehearsal added stage 3, a handover to the other arm, and both lessons, shown in blue. Text is quoted from the stored file, and omitted text is marked [...].

F Limitations and Future Work

Limitations. RoboScript relies on commercial foundation models served through APIs, whose service quality varies over time and can shift success by several points between sessions. Inference is also slow compared with a trained policy, since an evaluation episode with GPT-6.1-Sol takes on average 54.7 model calls and the robot waits for each call before it acts. Reflection improves the frozen memories in every round, but the net gain

(a) LESSON
added in rehearsal to the Task Memory of `move_stapler_pad`

Clean, same seeds
4/10 → 9/10

`L1` *When* grounding the stapler or verifying its released placement, especially when `check_done` passes but `on` or official completion fails, *do* use the live instruction's stapler and mat descriptions and `choose_arm` rule. Confirm the whole stapler includes its lower base, upper body and hinge before applying the pose grasp. After release, withdraw and remeasure the complete object from unobstructed views before checking `on` and `check_done`. Never finish on xy overlap alone: if `on` fails or finish is refused, verify the measured base/contact and containment before a reseating correction; do not repeatedly nudge from a stale or partial box.

(b) GUIDANCE
written by reflection for `handover_block`

Round 3, same seeds
4/10 → 10/10

`T-handover_block-02` (active; helpful 8, harmful 2; added r1)
Watch for abbreviated grab wording is interpreted as requiring a permanent donor hold, despite the empty blue pad and unfinished handover → Treat abbreviated grab wording as naming the donor stage, not the final state. If the pad is empty after grasping, continue the handover and released pad placement rather than stopping with the donor hold.

`T-handover_block-03` (active; helpful 6, harmful 0; added r3)
Instead of Task Memory step `place_on_pad`: *When* the receiver is lowering the block onto the pad, whose refined target may disagree with the visible blue edges → Lower using the stored pad placement without opening; verify the bottom lies inside the visible pad. If displaced, lift and recenter while retaining the receiver hold, then release and perform the stored clearance. [...]

Figure 16 Learned context that changes the outcome. Left, the source of each text and success on the same seeds without and with it. (a) The only lesson rehearsal adds to this Task Memory. Without it, all five lost episodes end with a finish on overlap alone. (b) Two guidance entries after round 3. Text is quoted, with cuts marked.

shrinks quickly as the easier failure modes are addressed first, and sparse success signals make the remaining failures hard to attribute. All evaluated tasks involve rigid or articulated objects. Deformable objects such as cloth and cables, and tasks that require tightly coordinated contact between the two arms such as handovers, remain challenging and call for richer perception, skills, and worlds.

Future work. Beyond these limitations, the agentic side of WICL remains largely unexplored. Agents could build new skills when existing ones fail, coordinate several specialized agents over a long task, or rehearse in learned world models when no simulator is available. Our results also suggest that agent-centric embodied models are a promising direction. A foundation model trained to use embodied tools, read physical feedback, and maintain a task memory could make WICL faster, cheaper, and more reliable than general-purpose models accessed through an API.

References

[1] Lakshya A Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziems, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J Ryan, Meng Jiang, Christopher Potts, Koushik Sen, Alexandros G. Dimakis, Ion Stoica, Dan Klein, Matei Zaharia, and Omar Khattab. GEPA: Reflective prompt evolution can outperform reinforcement learning. In *International Conference on Learning Representations*, pages 8479–8565, 2026. URL https://proceedings.iclr.cc/paper_files/paper/2026/file/0e9e708b6f48e14fd0ac29e167413f76-Paper-Conference.pdf.

[2] Kevin Black, Noah Brown, James Darpinian, Karan Dhabalia, Danny Driess, Adnan Esmail, Michael Robert Equi, Chelsea Finn, Niccolo Fusai, Manuel Y. Galliker, Dibya Ghosh, Lachy Groom, Karol Hausman, Brian Ichter, Szymon Jakubczak, Tim Jones, Liyiming Ke, Devin LeBlanc, Sergey Levine, Adrian Li-Bell, Mohith Mothukuri, Suraj Nair, Karl Pertsch, Allen Z. Ren, Lucy Xiaoyang Shi, Laura Smith, Jost Tobias Springenberg, Kyle Stachowicz, James Tanner, Quan Vuong, Homer Walke, Anna Walling, Haohuan Wang, Lili Yu, and Ury Zhilinsky. $\pi_{0.5}$: a vision-language-action model with open-world generalization. In Joseph Lim, Shuran Song, and Hae-Won Park, editors, *Proceedings of The 9th Conference on Robot Learning*, volume 305 of *Proceedings of Machine Learning Research*, pages 17–40. PMLR, September 2025. URL https://proceedings.mlr.press/v305/black25a.html.

[3] Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Robert Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, Karol Hausman, Brian Ichter, Szymon Jakubczak, Tim Jones, Liyiming Ke, Sergey Levine, Adrian Li-Bell, Mohith Mothukuri, Suraj Nair, Karl Pertsch, Lucy Xiaoyang Shi, Laura Smith, James Tanner, Quan Vuong, Anna Walling, Haohuan Wang, and Ury Zhilinsky. π_0: A Vision-Language-Action Flow Model for General Robot Control. In *Proceedings of Robotics: Science and Systems*, Los Angeles, CA, USA, June 2025. doi: 10.15607/RSS.2025.XXI.010. URL https://roboticsproceedings.org/rss21/p010.html.

[4] Konstantinos Bousmalis, Giulia Vezzani, Dushyant Rao, Coline Manon Devin, Alex X. Lee, Maria Bauza Villalonga, Todor Davchev, Yuxiang Zhou, Agrim Gupta, Akhil Raju, Antoine Laurens, Claudio Fantacci, Valentin Dalibard, Martina Zambelli, Murilo Fernandes Martins, Rugile Pevceviciute, Michiel Blokzijl, Misha Denil, Nathan Batchelor, Thomas Lampe, Emilio Parisotto, Konrad Zolna, Scott Reed, Sergio Gómez Colmenarejo, Jonathan Scholz, Abbas Abdolmaleki, Oliver Groth, Jean-Baptiste Regli, Oleg Sushkov, Thomas Rothörl, Jose Enrique Chen, Yusuf Aytar, David Barker, Joy Ortiz, Martin Riedmiller, Jost Tobias Springenberg, Raia Hadsell, Francesco Nori, and Nicolas Heess. RoboCat: A self-improving generalist agent for robotic manipulation. *Transactions on Machine Learning Research*, 2024. ISSN 2835-8856. URL https://openreview.net/forum?id=vsCpILiWHu.

[5] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel Ziegler, Jeffrey Wu, Clemens Winter, Chris Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners. In H. Larochelle, M. Ranzato, R. Hadsell, M.F. Balcan, and H. Lin, editors, *Advances in Neural Information Processing Systems*, volume 33, pages 1877–1901. Curran Associates, Inc., 2020.

[6] Qingwen Bu, Jisong Cai, Li Chen, Xiuqi Cui, Yan Ding, Siyuan Feng, Xindong He, Xu Huang, et al. AgiBot World Colosseo: A large-scale manipulation platform for scalable and intelligent embodied systems. In *2025 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 3549–3556, Hangzhou, China, 2025. IEEE. doi: 10.1109/IROS60139.2025.11247088.

[7] Jisong Cai, Long Ling, Shiwei Chu, Zhongshan Liu, Jiayue Kang, Zhixuan Liang, Wenjie Xu, Yinan Mao, Weinan Zhang, Xiaokang Yang, Ru Ying, Ran Zheng, and Yao Mu. AHA-WAM: Asynchronous horizon-adaptive world-action modeling with observation-guided context routing. *arXiv preprint arXiv:2606.09811*, 2026.

[8] Rui Cai, Jun Guo, Xinze He, Piaopiao Jin, Jie Li, Bingxuan Lin, Futeng Liu, Wei Liu, Fei Ma, Kun Ma, Feng Qiu, Heng Qu, Yifei Su, Qiao Sun, Dong Wang, Donghao Wang, Yunhong Wang, Rujie Wu, Diyun Xiang, Yu Yang, Hangjun Ye, Yuan Zhang, and Quanyun Zhou. Xiaomi-robotics-0: An open-sourced vision-language-action model with real-time execution. *arXiv preprint arXiv:2602.12684*, 2026.

[9] Chi-Lam Cheang, Guangzeng Chen, Ya Jing, Tao Kong, Hang Li, Yifeng Li, Yuxiao Liu, Hongtao Wu, Jiafeng Xu, Yichu Yang, Hanbo Zhang, and Minzhao Zhu. GR-2: A generative video-language-action model with web-scale knowledge for robot manipulation. *arXiv preprint arXiv:2410.06158*, 2024.

[10] Siyi Chen, Hugo Hadfield, Alex Zook, Mikaela Angelina Uy, Chan Hee Song, Erwin Coumans, Xuning Yang,

Faisal Ladhak, Qing Qu, Stan Birchfield, Jonathan Tremblay, and Valts Blukis. VoLo: A physical orchestrator for open-vocabulary long-horizon manipulation. In *Conference on Robot Learning (CoRL)*, 2026. URL https://openreview.net/forum?id=sF3YZbbTNX.

[11] Tianxing Chen, Zanxin Chen, Baijun Chen, Zijian Cai, Yibin Liu, Zixuan Li, Qiwei Liang, Xianliang Lin, Yiheng Ge, Zhenyu Gu, Weiliang Deng, Yubin Guo, Tian Nian, Xuanbing Xie, Qiangyu Chen, Kailun Su, Tianling Xu, Guodong Liu, Mengkang Hu, Huan-Ang Gao, Kaixuan Wang, Zhixuan Liang, Yusen Qin, Xiaokang Yang, Ping Luo, and Yao Mu. RoboTwin 2.0: A scalable data generator and benchmark with strong domain randomization for robust bimanual robotic manipulation. In Tong Zhang, Miroslav Dudik, Martin Jaggi, Alekh Agarwal, Sharon Li, Dale Schuurmans, Jerry Zhu, Felix Berkenkamp, Hanze Dong, and Alberto Bietti, editors, *Proceedings of the 43rd International Conference on Machine Learning*, volume 306 of *Proceedings of Machine Learning Research*, pages 13673–13699. PMLR, 2026. URL https://proceedings.mlr.press/v306/chen26j.html.

[12] Yixiang Chen, Peiyan Li, Yuan Xu, Qisen Ma, Jiabing Yang, Kai Wang, Jianhua Yang, Dong An, He Guan, Gaoteng Liu, Jianlou Si, Jun Huang, Jing Liu, Nianfeng Liu, Yan Huang, and Liang Wang. FlowWAM: Optical flow as a unified action representation for world action models. *arXiv preprint arXiv:2607.13017*, 2026. URL https://arxiv.org/abs/2607.13017.

[13] Norman Di Palo and Edward Johns. Keypoint action tokens enable in-context imitation learning in robotics. In *Proceedings of Robotics: Science and Systems*, Delft, Netherlands, July 2024. doi: 10.15607/RSS.2024.XX.096.

[14] Qingxiu Dong, Lei Li, Damai Dai, Ce Zheng, Jingyuan Ma, Rui Li, Heming Xia, Jingjing Xu, Zhiyong Wu, Baobao Chang, Xu Sun, Lei Li, and Zhifang Sui. A survey on in-context learning. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 1107–1128, Miami, Florida, USA, November 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.emnlp-main.64.

[15] Danny Driess, Fei Xia, Mehdi S. M. Sajjadi, Corey Lynch, Aakanksha Chowdhery, Brian Ichter, Ayzaan Wahid, Jonathan Tompson, Quan Vuong, Tianhe Yu, Wenlong Huang, Yevgen Chebotar, Pierre Sermanet, Daniel Duckworth, Sergey Levine, Vincent Vanhoucke, Karol Hausman, Marc Toussaint, Klaus Greff, Andy Zeng, Igor Mordatch, and Pete Florence. PaLM-e: An embodied multimodal language model. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 8469–8488. PMLR, 2023. URL https://proceedings.mlr.press/v202/driess23a.html.

[16] Yilun Du, Sherry Yang, Bo Dai, Hanjun Dai, Ofir Nachum, Josh Tenenbaum, Dale Schuurmans, and Pieter Abbeel. Learning universal policies via text-guided video generation. In *Advances in Neural Information Processing Systems*, volume 36, pages 9156–9172, 2023. doi: 10.52202/075280-0403.

[17] Yan Duan, Marcin Andrychowicz, Bradly Stadie, Jonathan Ho, Jonas Schneider, Ilya Sutskever, Pieter Abbeel, and Wojciech Zaremba. One-shot imitation learning. In *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc., 2017. URL https://proceedings.neurips.cc/paper_files/paper/2017/file/ba3866600c3540f67c1e9575e213be0a-Paper.pdf.

[18] Heng Fang, Shangru Li, Shuhan Wang, Xuanyang Xi, Dingkang Liang, and Xiang Bai. Towards generalizable robotic manipulation in dynamic environments. In *European Conference on Computer Vision*, pages 314–333. Springer, 2026.

[19] Kuan Fang, Fangchen Liu, Pieter Abbeel, and Sergey Levine. MOKA: Open-world robotic manipulation through mark-based visual prompting. In *Proceedings of Robotics: Science and Systems*, Delft, Netherlands, July 2024. doi: 10.15607/RSS.2024.XX.062.

[20] Chelsea Finn, Tianhe Yu, Tianhao Zhang, Pieter Abbeel, and Sergey Levine. One-shot visual imitation learning via meta-learning. In Sergey Levine, Vincent Vanhoucke, and Ken Goldberg, editors, *Proceedings of the 1st Annual Conference on Robot Learning*, volume 78 of *Proceedings of Machine Learning Research*, pages 357–368. PMLR, 2017. URL https://proceedings.mlr.press/v78/finn17a.html.

[21] Haoyu Fu, Diankun Zhang, Zongchuang Zhao, Jianfeng Cui, Dingkang Liang, Chong Zhang, Dingyuan Zhang, Hongwei Xie, Bing Wang, and Xiang Bai. Orion: A holistic end-to-end autonomous driving framework by vision-language instructed action generation. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2025.

[22] Haoyu Fu, Diankun Zhang, Zongchuang Zhao, Jianfeng Cui, Hongwei Xie, Bing Wang, Guang Chen, Dingkang Liang, and Xiang Bai. Minddrive: A vision-language-action model for autonomous driving via online reinforcement learning. In *Proceedings of the European Conference on Computer Vision*, 2026.

[23] Letian Fu, Justin Yu, Karim El-Refai, Ethan Kou, Haoru Xue, Huang Huang, Wenli Xiao, Li Fei-Fei, Guanya Shi, Jiajun Wu, S. Shankar Sastry, Yuke Zhu, Ken Goldberg, and Linxi Fan. CaP-X: A framework for benchmarking and improving coding agents for robot manipulation. In *Proceedings of the 43rd International Conference on Machine Learning*, volume 306 of *Proceedings of Machine Learning Research*, pages 32155–32212. PMLR, 2026. URL https://proceedings.mlr.press/v306/fu26u.html.

[24] Max Fu, Huang Huang, Gaurav Datta, Lawrence Yunliang Chen, Will Panitch, Fangchen Liu, Hui Li, and Ken Goldberg. ICRT: In-context imitation learning via next-token prediction. In *2025 IEEE International Conference on Robotics and Automation (ICRA)*, pages 5937–5944. IEEE, 2025. doi: 10.1109/ICRA55743.2025.11128272.

[25] Ning Gao, Jinliang Zheng, Xing Gao, Haoxiang Ma, Hanqing Wang, Yukai Wang, Jiantong Chen, Zanxin Chen, Shujie Zhang, Mingda Jia, Xuekun Jiang, Zihou Zhu, Xinyu Li, Shuai Wang, Hao Li, Wenzhe Cai, Yuqiang Yang, Xudong Xu, Zhaoyang Lyu, Yao Mu, Tai Wang, Jiangmiao Pang, Jia Zeng, Weinan Zhang, and Chunhua Shen. EBench: Elemental diagnosis of generalist mobile manipulation policies. *arXiv preprint arXiv:2606.18239*, 2026.

[26] Generalist Team. GEN-1.5: Embodied foundation models are one-shot learners. Generalist AI Blog, https://generalistai.com/blog/gen-1.5, August 2026.

[27] Zhaopeng Gu, Bingke Zhu, Tianxi Lin, Guibo Zhu, Yingying Chen, Kai Wang, Tingyu Yuan, Fan Yang, Chaoyang Zhao, Zhaowen Li, Peng Su, and Jinqiao Wang. HarnessWAM: Bridging prediction and deliberation in world action models. *arXiv preprint arXiv:2608.09516*, 2026.

[28] Jun Guo, Qiwei Li, Peiyan Li, Zilong Chen, Nan Sun, Yifei Su, Heyun Wang, Yuan Zhang, Xinghang Li, and Huaping Liu. Unified 4D world action modeling from video priors with asynchronous denoising. *arXiv preprint arXiv:2604.26694*, 2026.

[29] Yanjiang Guo, Lucy Xiaoyang Shi, Jianyu Chen, and Chelsea Finn. Ctrl-World: A controllable generative world model for robot manipulation. In *International Conference on Learning Representations*, pages 6121–6138, 2026. URL https://proceedings.iclr.cc/paper_files/paper/2026/file/0ae94013da7cd459402fd77874e09ee3-Paper-Conference.pdf.

[30] Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. Mastering diverse control tasks through world models. *Nature*, 640(8059):647–653, 2025. doi: 10.1038/s41586-025-08744-2.

[31] Jinbang Huang, Zhiyuan Li, Yuanzhao Hu, Ran Qi, Yixin Xiao, Mark Coates, Tongtong Cao, Zhanguang Zhang, and Yingxue Zhang. RoboHarness: Memory-driven orchestration of heterogeneous robot policies for long-horizon planning. *arXiv preprint arXiv:2607.18060*, 2026.

[32] Wenlong Huang, Chen Wang, Ruohan Zhang, Yunzhu Li, Jiajun Wu, and Fei-Fei Li. VoxPoser: Composable 3D value maps for robotic manipulation with language models. In Jie Tan, Marc Toussaint, and Kourosh Darvish, editors, *Proceedings of The 7th Conference on Robot Learning*, volume 229 of *Proceedings of Machine Learning Research*, pages 540–562. PMLR, 2023. URL https://proceedings.mlr.press/v229/huang23b.html.

[33] Wenlong Huang, Fei Xia, Ted Xiao, Harris Chan, Jacky Liang, Pete Florence, Andy Zeng, Jonathan Tompson, Igor Mordatch, Yevgen Chebotar, Pierre Sermanet, Tomas Jackson, Noah Brown, Linda Luu, Sergey Levine, Karol Hausman, and Brian Ichter. Inner monologue: Embodied reasoning through planning with language models. In Karen Liu, Dana Kulic, and Jeff Ichnowski, editors, *Proceedings of The 6th Conference on Robot Learning*, volume 205 of *Proceedings of Machine Learning Research*, pages 1769–1782. PMLR, 2023. URL https://proceedings.mlr.press/v205/huang23c.html.

[34] Wenlong Huang, Chen Wang, Yunzhu Li, Ruohan Zhang, and Li Fei-Fei. Rekep: Spatio-temporal reasoning of relational keypoint constraints for robotic manipulation. In Pulkit Agrawal, Oliver Kroemer, and Wolfram Burgard, editors, *Proceedings of The 8th Conference on Robot Learning*, volume 270 of *Proceedings of Machine Learning Research*, pages 4573–4602. PMLR, 2025. URL https://proceedings.mlr.press/v270/huang25g.html.

[35] Brian Ichter, Anthony Brohan, Yevgen Chebotar, Chelsea Finn, Karol Hausman, Alexander Herzog, Daniel Ho, Julian Ibarz, Alex Irpan, Eric Jang, Ryan Julian, Dmitry Kalashnikov, Sergey Levine, Yao Lu, Carolina Parada, Kanishka Rao, Pierre Sermanet, Alexander T Toshev, Vincent Vanhoucke, Fei Xia, Ted Xiao, Peng Xu, Mengyuan Yan, Noah Brown, Michael Ahn, Omar Cortes, Nicolas Sievers, Clayton Tan, Sichun Xu, Diego Reyes, Jarek Rettinghouse, Jornell Quiambao, Peter Pastor, Linda Luu, Kuang-Huei Lee, Yuheng Kuang, Sally Jesmonth, Nikhil J. Joshi, Kyle Jeffrey, Rosario Jauregui Ruano, Jasmine Hsu, Keerthana Gopalakrishnan, Byron David, Andy Zeng, and Chuyuan Kelly Fu. Do as i can, not as i say: Grounding language in robotic affordances. In Karen Liu, Dana Kulic, and Jeff Ichnowski, editors, *Proceedings of The 6th Conference on*

Robot Learning, volume 205 of *Proceedings of Machine Learning Research*, pages 287–318. PMLR, 2023. URL https://proceedings.mlr.press/v205/ichter23a.html.

[36] Stephen James, Michael Bloesch, and Andrew J. Davison. Task-embedded control networks for few-shot imitation learning. In Aude Billard, Anca Dragan, Jan Peters, and Jun Morimoto, editors, *Proceedings of The 2nd Conference on Robot Learning*, volume 87 of *Proceedings of Machine Learning Research*, pages 783–795. PMLR, 2018. URL https://proceedings.mlr.press/v87/james18a.html.

[37] Joel Jang, Seonghyeon Ye, Zongyu Lin, Jiannan Xiang, Johan Bjorck, Yu Fang, Fengyuan Hu, Spencer Huang, Kaushil Kundalia, Yen-Chen Lin, Loïc Magne, Ajay Mandlekar, Avnish Narayan, You Liang Tan, Guanzhi Wang, Jing Wang, Qi Wang, Yinzhen Xu, Xiaohui Zeng, Kaiyuan Zheng, Ruijie Zheng, Ming-Yu Liu, Luke Zettlemoyer, Dieter Fox, Jan Kautz, Scott Reed, Yuke Zhu, and Linxi Fan. Dreamgen: Unlocking generalization in robot learning through video world models. In *Proceedings of The 9th Conference on Robot Learning*, volume 305 of *Proceedings of Machine Learning Research*, pages 5170–5194. PMLR, 2025.

[38] Tao Jiang, Tianyuan Yuan, Yicheng Liu, Chenhao Lu, Jianning Cui, Xiao Liu, Shuiqi Cheng, Jiyang Gao, Huazhe Xu, and Hang Zhao. Galaxy open-world dataset and G0 dual-system VLA model. In *2026 IEEE International Conference on Robotics and Automation (ICRA)*, pages 20749–20756, Vienna, Austria, 2026. IEEE. doi: 10.1109/ICRA57385.2026.11696535.

[39] Dmitry Kalashnikov, Alex Irpan, Peter Pastor, Julian Ibarz, Alexander Herzog, Eric Jang, Deirdre Quillen, Ethan Holly, Mrinal Kalakrishnan, Vincent Vanhoucke, and Sergey Levine. Scalable deep reinforcement learning for vision-based robotic manipulation. In Aude Billard, Anca Dragan, Jan Peters, and Jun Morimoto, editors, *Proceedings of The 2nd Conference on Robot Learning*, volume 87 of *Proceedings of Machine Learning Research*, pages 651–673. PMLR, 2018.

[40] Alexander Khazatsky, Karl Pertsch, Suraj Nair, Ashwin Balakrishna, Sudeep Dasari, Siddharth Karamcheti, Soroush Nasiriany, Mohan Kumar Srirama, Lawrence Yunliang Chen, Kirsty Ellis, Peter David Fagan, Joey Hejna, Masha Itkina, Marion Lepert, Yecheng Jason Ma, Patrick Tree Miller, Jimmy Wu, Suneel Belkhale, Shivin Dass, Huy Ha, Arhan Jain, Abraham Lee, Youngwoon Lee, Marius Memmel, Sungjae Park, Ilija Radosavovic, Kaiyuan Wang, Albert Zhan, Kevin Black, Cheng Chi, Kyle Beltran Hatch, Shan Lin, Jingpei Lu, Jean Mercat, Abdul Rehman, Pannag R Sanketi, Archit Sharma, Cody Simpson, Quan Vuong, Homer Rich Walke, Blake Wulfe, Ted Xiao, Jonathan Heewon Yang, Arefeh Yavary, Tony Z. Zhao, Christopher Agia, Rohan Baijal, Mateo Guaman Castro, Daphne Chen, Qiuyu Chen, Trinity Chung, Jaimyn Drake, Ethan Paul Foster, Jensen Gao, David Antonio Herrera, Minho Heo, Kyle Hsu, Jiaheng Hu, Donovon Jackson, Charlotte Le, Yunshuang Li, Roy Lin, Zehan Ma, Abhiram Maddukuri, Suvir Mirchandani, Daniel Morton, Tony Nguyen, Abigail O'Neill, Rosario Scalise, Derick Seale, Victor Son, Stephen Tian, Emi Tran, Andrew E. Wang, Yilin Wu, Annie Xie, Jingyun Yang, Patrick Yin, Yunchu Zhang, Osbert Bastani, Glen Berseth, Jeannette Bohg, Ken Goldberg, Abhinav Gupta, Abhishek Gupta, Dinesh Jayaraman, Joseph J Lim, Jitendra Malik, Roberto Martín-Martín, Subramanian Ramamoorthy, Dorsa Sadigh, Shuran Song, Jiajun Wu, Michael C. Yip, Yuke Zhu, Thomas Kollar, Sergey Levine, and Chelsea Finn. DROID: A Large-Scale In-The-Wild Robot Manipulation Dataset. In *Proceedings of Robotics: Science and Systems*, Delft, Netherlands, July 2024. doi: 10.15607/RSS.2024.XX.120. URL https://roboticsproceedings.org/rss20/p120.html.

[41] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan P Foster, Pannag R Sanketi, Quan Vuong, Thomas Kollar, Benjamin Burchfiel, Russ Tedrake, Dorsa Sadigh, Sergey Levine, Percy Liang, and Chelsea Finn. OpenVLA: An open-source vision-language-action model. In Pulkit Agrawal, Oliver Kroemer, and Wolfram Burgard, editors, *Proceedings of The 8th Conference on Robot Learning*, volume 270 of *Proceedings of Machine Learning Research*, pages 2679–2713. PMLR, 2025. URL https://proceedings.mlr.press/v270/kim25c.html.

[42] Moo Jin Kim, Yihuai Gao, Tsung-Yi Lin, Yen-Chen Lin, Yunhao Ge, Grace Lam, Percy Liang, Shuran Song, Ming-Yu Liu, Chelsea Finn, and Jinwei Gu. Cosmos policy: Fine-tuning video models for visuomotor control and planning. In *International Conference on Learning Representations*, pages 71531–71552, 2026. URL https://proceedings.iclr.cc/paper_files/paper/2026/file/748becc400a57c0e31cfe6a2e7951467-Paper-Conference.pdf.

[43] Michael Laskin, Luyu Wang, Junhyuk Oh, Emilio Parisotto, Stephen Spencer, Richie Steigerwald, DJ Strouse, Steven Hansen, Angelos Filos, Ethan Brooks, Maxime Gazeau, Himanshu Sahni, Satinder Singh, and Volodymyr Mnih. In-context reinforcement learning with algorithm distillation. In *The Eleventh International Conference on Learning Representations*, 2023.

[44] Jonathan N. Lee, Annie Xie, Aldo Pacchiano, Yash Chandak, Chelsea Finn, Ofir Nachum, and Emma Brunskill. Supervised pretraining can learn in-context reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 36, pages 43057–43083, 2023. doi: 10.52202/075280-1866.

[45] Fuhao Li, Wenxuan Song, Han Zhao, Jingbo Wang, Pengxiang Ding, Donglin Wang, Long Zeng, and Haoang Li. Spatial forcing: Implicit spatial representation alignment for vision-language-action model. In *International Conference on Learning Representations*, volume 2026, pages 132324–132345, 2026.

[46] Haozhan Li, Yuxin Zuo, Jiale Yu, Yuhao Zhang, Zhaohui Yang, Kaiyan Zhang, Xuekai Zhu, Yuchen Zhang, Tianxing Chen, Ganqu Cui, Dehui Wang, Dingxiang Luo, Yuchen Fan, Youbang Sun, Jia Zeng, Jiangmiao Pang, Shanghang Zhang, Yu Wang, Yao Mu, Bowen Zhou, and Ning Ding. SimpleVLA-RL: Scaling VLA training via reinforcement learning. In *International Conference on Learning Representations*, pages 125846–125869, 2026. URL https://proceedings.iclr.cc/paper_files/paper/2026/file/cbfbcb4da14235bd69b134070898ae9d-Paper-Conference.pdf.

[47] Lin Li, Qihang Zhang, Yiming Luo, Shuai Yang, Ruilin Wang, Luyao Zhang, Mingrui Yu, Zelin Gao, Nan Xue, Boyu Zhou, Xing Zhu, Mingyu Ding, Yujun Shen, and Yinghao Xu. Causal World Modeling for Robot Control. In *Proceedings of Robotics: Science and Systems*, Sydney, Australia, July 2026. doi: 10.15607/RSS.2026.XXII.016. URL https://roboticsproceedings.org/rss22/p016.html.

[48] Yaxuan Li, Yichen Zhu, Junjie Wen, Chaomin Shen, and Yi Xu. WorldEval: World model as real-world robot policies evaluator. *arXiv preprint arXiv:2505.19017*, 2025.

[49] Dingkang Liang, Xin Zhou, Wei Xu, Xingkui Zhu, Zhikang Zou, Xiaoqing Ye, Xiao Tan, and Xiang Bai. Pointmamba: A simple state space model for point cloud analysis. In *Advances in Neural Information Processing Systems*, 2024.

[50] Dingkang Liang, Tianrui Feng, Xin Zhou, Yumeng Zhang, Zhikang Zou, and Xiang Bai. Parameter-efficient fine-tuning in spectral domain for point cloud learning. *IEEE transactions on pattern analysis and machine intelligence*, 2025.

[51] Dingkang Liang, Cheng Zhang, Xiaopeng Xu, Jianzhong Ju, Zhenbo Luo, and Xiang Bai. Cook and clean together: Teaching embodied agents for parallel task execution. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2026.

[52] Dingkang Liang, Dingyuan Zhang, Xin Zhou, Sifan Tu, Tianrui Feng, Xiaofan Li, Yumeng Zhang, Mingyang Du, Xiao Tan, and Xiang Bai. Unifuture: A 4d driving world model for future generation and perception. In *IEEE International Conference on Robotics and Automation*, 2026.

[53] Jacky Liang, Wenlong Huang, Fei Xia, Peng Xu, Karol Hausman, Brian Ichter, Pete Florence, and Andy Zeng. Code as policies: Language model programs for embodied control. In *2023 IEEE International Conference on Robotics and Automation (ICRA)*, pages 9493–9500. IEEE, 2023. doi: 10.1109/ICRA48891.2023.10160591.

[54] Haowen Liu, Xirui Li, Shaoxiong Yao, Peng Shi, Tianyi Zhou, Jia-Bin Huang, Furong Huang, and Jiayuan Mao. Guava: Distilling frontier VLMs into a compact agent through a robotic manipulation harness. *arXiv preprint arXiv:2606.18363*, 2026.

[55] Songming Liu, Lingxuan Wu, Bangguo Li, Hengkai Tan, Huayu Chen, Zhengyi Wang, Ke Xu, Hang Su, and Jun Zhu. RDT-1B: a diffusion foundation model for bimanual manipulation. In *International Conference on Learning Representations*, volume 2025, pages 29982–30009, 2025. URL https://proceedings.iclr.cc/paper_files/paper/2025/hash/49f80e4d2471ad4f2edf4f5f1ab62339-Abstract-Conference.html.

[56] Jianlan Luo, Zheyuan Hu, Charles Xu, You Liang Tan, Jacob Berg, Archit Sharma, Stefan Schaal, Chelsea Finn, Abhishek Gupta, and Sergey Levine. SERL: A software suite for sample-efficient robotic reinforcement learning. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pages 16961–16969. IEEE, 2024. doi: 10.1109/ICRA57147.2024.10610040.

[57] Jianlan Luo, Charles Xu, Jeffrey Wu, and Sergey Levine. Precise and dexterous robotic manipulation via human-in-the-loop reinforcement learning. *Science Robotics*, 10(105):eads5033, 2025. doi: 10.1126/scirobotics.ads5033.

[58] Yecheng Jason Ma, William Liang, Guanzhi Wang, De-An Huang, Osbert Bastani, Dinesh Jayaraman, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Eureka: Human-level reward design via coding large language models. In *International Conference on Learning Representations*, volume 2024, pages 26516–26560, 2024.

[59] NVIDIA, Johan Bjorck, Fernando Castañeda, Nikita Cherniadev, Xingye Da, Runyu Ding, Linxi "Jim" Fan, Yu Fang, Dieter Fox, Fengyuan Hu, Spencer Huang, Joel Jang, Zhenyu Jiang, Jan Kautz, Kaushil Kundalia, Lawrence Lao, Zhiqi Li, Zongyu Lin, Kevin Lin, Guilin Liu, Edith Llontop, Loic Magne, Ajay Mandlekar, Avnish Narayan, Soroush Nasiriany, Scott Reed, You Liang Tan, Guanzhi Wang, Zu Wang, Jing Wang, Qi Wang, Jiannan Xiang, Yuqi Xie, Yinzhen Xu, Zhenjia Xu, Seonghyeon Ye, Zhiding Yu, Ao Zhang, Hao Zhang, Yizhou Zhao, Ruijie Zheng, and Yuke Zhu. GR00T N1: An open foundation model for generalist humanoid robots. *arXiv preprint arXiv:2503.14734*, 2025. URL https://arxiv.org/abs/2503.14734.

[60] Hanbit Oh, Andrea M. Salcedo-Vázquez, Ixchel G. Ramirez-Alpizar, and Yukiyasu Domae. Robust instant policy: Leveraging student's t-regression model for robust in-context imitation learning of robot manipulation. In *2025 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 7973–7980. IEEE, 2025. doi: 10.1109/IROS60139.2025.11247005.

[61] Open X-Embodiment Collaboration. Open X-Embodiment: Robotic learning datasets and RT-X models. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pages 6892–6903, Yokohama, Japan, 2024. IEEE. doi: 10.1109/ICRA57147.2024.10611477. URL https://doi.org/10.1109/ICRA57147.2024.10611477.

[62] OpenAI, Marcin Andrychowicz, Bowen Baker, Maciek Chociej, Rafal Józefowicz, Bob McGrew, Jakub Pachocki, Arthur Petron, Matthias Plappert, Glenn Powell, Alex Ray, Jonas Schneider, Szymon Sidor, Josh Tobin, Peter Welinder, Lilian Weng, and Wojciech Zaremba. Learning dexterous in-hand manipulation. *The International Journal of Robotics Research*, 39(1):3–20, 2020. doi: 10.1177/0278364919887447.

[63] Alessio Palma, Indro Spinelli, Vignesh Prasad, Luca Scofano, Yufeng Jin, Georgia Chalvatzaki, and Fabio Galasso. Bimanual robot manipulation via multi-agent in-context learning. In *Conference on Robot Learning (CoRL)*, 2026. URL https://openreview.net/forum?id=bLQynefrlp.

[64] Seyed Kamyar Seyed Ghasemipour, Ayzaan Wahid, Jonathan Tompson, Pannag Sanketi, and Igor Mordatch. Self-improving embodied foundation models. In *Advances in Neural Information Processing Systems*, volume 38, 2025. doi: 10.52202/085713-3762.

[65] Lucy Xiaoyang Shi, Brian Ichter, Michael Robert Equi, Liyiming Ke, Karl Pertsch, Quan Vuong, James Tanner, Anna Walling, Haohuan Wang, Niccolo Fusai, Adrian Li-Bell, Danny Driess, Lachy Groom, Sergey Levine, and Chelsea Finn. Hi robot: Open-ended instruction following with hierarchical vision-language-action models. In Aarti Singh, Maryam Fazel, Daniel Hsu, Simon Lacoste-Julien, Felix Berkenkamp, Tegan Maharaj, Kiri Wagstaff, and Jerry Zhu, editors, *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 54919–54933. PMLR, 2025. URL https://proceedings.mlr.press/v267/shi25d.html.

[66] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 36, pages 8634–8652, 2023. doi: 10.52202/075280-0377.

[67] Ishika Singh, Valts Blukis, Arsalan Mousavian, Ankit Goyal, Danfei Xu, Jonathan Tremblay, Dieter Fox, Jesse Thomason, and Animesh Garg. ProgPrompt: Generating situated robot task plans using large language models. In *2023 IEEE International Conference on Robotics and Automation (ICRA)*, pages 11523–11530. IEEE, 2023. doi: 10.1109/ICRA48891.2023.10161317.

[68] Skild AI. Introducing S1: In-context learning for robotics. Skild AI blog, https://www.skild.ai/blogs/s1, August 2026.

[69] Kaustubh Sridhar, Souradeep Dutta, Dinesh Jayaraman, and Insup Lee. RICL: Adding in-context adaptability to pre-trained vision-language-action models. In Joseph Lim, Shuran Song, and Hae-Won Park, editors, *Proceedings of The 9th Conference on Robot Learning*, volume 305 of *Proceedings of Machine Learning Research*, pages 5022–5038. PMLR, 2025.

[70] StarVLA Community. Starvla: A lego-like codebase for vision-language-action model developing. *arXiv preprint arXiv:2604.05014*, 2026.

[71] Marcel Torne Villasevil, Arhan Jain, Jiayi Yuan, Vidyaaranya Macha, Lars Lien Ankile, Anthony Simeonov, Pulkit Agrawal, and Abhishek Gupta. Robot learning with super-linear scaling. In *Proceedings of Robotics: Science and Systems*, Los Angeles, CA, USA, June 2025. doi: 10.15607/RSS.2025.XXI.025.

[72] Brian Y. Tsui, Alan Y. Fang, and Tiffany J. Hwu. Demonstration-free robotic control via LLM agents. In

2026 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), 2026. URL https://arxiv.org/abs/2601.20334.

[73] Vitalis Vosylius and Edward Johns. Instant policy: In-context imitation learning via graph diffusion. In *International Conference on Learning Representations (ICLR)*, pages 35011–35030, 2025. URL https://proceedings.iclr.cc/paper_files/paper/2025/hash/5692c7dbc4abcaa50f9ce609819212e5-Abstract-Conference.html.

[74] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *Transactions on Machine Learning Research*, 2024. ISSN 2835-8856. URL https://openreview.net/forum?id=ehfRiF0R3a.

[75] Qi Wang, Tianyi Wang, Chengyang Li, Shikun Ban, Yurun Chen, Yizhong Ge, Jason Qin, Chengtai Li, and Wentao Zhu. Towards the harness of embodied agents. *arXiv preprint arXiv:2608.11246*, 2026.

[76] Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, Hoang Tran, Fuqiang Li, Ren Ma, Mingzhang Zheng, Bill Qian, Daniel Shao, Niklas Muennighoff, Yizhe Zhang, Binyuan Hui, Junyang Lin, Robert Brennan, Hao Peng, Heng Ji, and Graham Neubig. OpenHands: An open platform for AI software developers as generalist agents. In *International Conference on Learning Representations*, volume 2025, pages 65882–65919, 2025. URL https://proceedings.iclr.cc/paper_files/paper/2025/hash/a4b6ad6b48850c0c331d1259fc66a69c-Abstract-Conference.html.

[77] Zora Zhiruo Wang, Jiayuan Mao, Daniel Fried, and Graham Neubig. Agent workflow memory. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 63897–63911. PMLR, 2025.

[78] Philipp Wu, Alejandro Escontrela, Danijar Hafner, Pieter Abbeel, and Ken Goldberg. DayDreamer: World models for physical robot learning. In Karen Liu, Dana Kulic, and Jeff Ichnowski, editors, *Proceedings of The 6th Conference on Robot Learning*, volume 205 of *Proceedings of Machine Learning Research*, pages 2226–2240. PMLR, 2023.

[79] Wei Wu, Fan Lu, Yunnan Wang, Shuai Yang, Shi Liu, Fangjing Wang, Qian Zhu, He Sun, Yong Wang, Shuailei Ma, Yiyu Ren, Kejia Zhang, Hui Yu, Jingmei Zhao, Shuai Zhou, Zhenqi Qiu, Houlong Xiong, Ziyu Wang, Zechen Wang, Ran Cheng, Yong-Lu Li, Yongtao Huang, Xing Zhu, Yujun Shen, and Kecheng Zheng. A pragmatic VLA foundation model. *arXiv preprint arXiv:2601.18692*, 2026. URL https://arxiv.org/abs/2601.18692.

[80] Xianjin Wu, Dingkang Liang, Tianrui Feng, Kui Xia, Yumeng Zhang, Xiaofan Li, Xiao Tan, and Xiang Bai. Generation models know space: Unleashing implicit 3d priors for scene understanding. In *Proceedings of the European Conference on Computer Vision (ECCV)*, 2026.

[81] Hengyi Xie, Chenfei Yao, Xianjin Wu, Xuanyang Xi, Yiping Tang, Di Xu, Yingying Zhu, Dingkang Liang, Xiang Bai, and Han Ding. Turbovla: Real-time vision-language-action model at 32 hz on an rtx 4090 with $<$1 gb vram. *arXiv preprint arXiv:2607.27205*, 2026.

[82] Siyu Xu, Yunke Wang, Zijian Wang, Dihao Zhu, Chenghao Xia, Chengbin Du, Daochang Liu, Tao Huang, and Chang Xu. StellaVLA: In-context structured demonstration for generalizable vision-language-action models. *arXiv preprint arXiv:2608.11671*, 2026.

[83] John Yang, Carlos Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang, editors, *Advances in Neural Information Processing Systems*, volume 37, pages 50528–50652. Curran Associates, Inc., 2024. doi: 10.52202/079017-1601. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/5a7c947568c1b1328ccc5230172e1e7c-Abstract-Conference.html.

[84] Lishan Yang, Wenxuan Song, Xi Wang, Pingyue Sheng, Zheng Fang, Ziyang Zhou, Junjie He, Haodong Yan, Jiayi Chen, Nan Sun, Qiao Sun, Pengwei Wang, Lingqiao Liu, Yan Wang, Yuxiang Gao, Feras Dayoub, and Haoang Li. 4d-wam: Infusing spatiotemporal awareness into world action models through trajectory fields, 2026. URL https://arxiv.org/abs/2608.08023.

[85] Yandan Yang, Shuang Zeng, Tong Lin, Xinyuan Chang, Dekang Qi, Junjin Xiao, Haoyun Liu, Ronghan Chen, Yuzhi Chen, Dongjie Huo, Feng Xiong, Xing Wei, Zhiheng Ma, and Mu Xu. Abot-m0: Vla foundation model for robotic manipulation with action manifold learning. *arXiv preprint arXiv:2602.11236*, 2026.

[86] Seonghyeon Ye, Yunhao Ge, Kaiyuan Zheng, Shenyuan Gao, Sihyun Yu, George Kurian, Suneel Indupuru, You Liang Tan, Chuning Zhu, Jiannan Xiang, Ayaan Malik, Kyungmin Lee, William Liang, Nadun Ranawaka,

Jiasheng Gu, Yinzhen Xu, Guanzhi Wang, Fengyuan Hu, Avnish Narayan, Johan Bjorck, Jing Wang, Gwanghyun Kim, Dantong Niu, Ruijie Zheng, Yuqi Xie, Jimmy Wu, Qi Wang, Ryan Julian, Danfei Xu, Yilun Du, Yevgen Chebotar, Scott Reed, Jan Kautz, Yuke Zhu, Linxi "Jim" Fan, and Joel Jang. World action models are zero-shot policies. In *Conference on Robot Learning (CoRL)*, 2026. URL https://openreview.net/forum?id=Bshs9HDDiu.

[87] Yida Yin, Zekai Wang, Yuvan Sharma, Dantong Niu, Trevor Darrell, and Roei Herzig. In-context learning enables robot action prediction in LLMs. In *2025 IEEE International Conference on Robotics and Automation (ICRA)*, pages 8972–8979, Atlanta, GA, USA, 2025. IEEE. doi: 10.1109/ICRA55743.2025.11128807.

[88] Wenhao Yu, Nimrod Gileadi, Chuyuan Fu, Sean Kirmani, Kuang-Huei Lee, Montserrat Gonzalez Arenas, Hao-Tien Lewis Chiang, Tom Erez, Leonard Hasenclever, Jan Humplik, Brian Ichter, Ted Xiao, Peng Xu, Andy Zeng, Tingnan Zhang, Nicolas Heess, Dorsa Sadigh, Jie Tan, Yuval Tassa, and Fei Xia. Language to rewards for robotic skill synthesis. In Jie Tan, Marc Toussaint, and Kourosh Darvish, editors, *Proceedings of The 7th Conference on Robot Learning*, volume 229 of *Proceedings of Machine Learning Research*, pages 374–404. PMLR, 2023. URL https://proceedings.mlr.press/v229/yu23a.html.

[89] Tianyuan Yuan, Zibin Dong, Yicheng Liu, and Hang Zhao. Fast-WAM: Do world action models need test-time future imagination? *arXiv preprint arXiv:2603.16666*, 2026.

[90] Borong Zhang, Jiahao Li, Jiachen Shen, Yuhao Zhang, Yishuai Cai, Yuanpei Chen, Juntao Dai, Jiaming Ji, and Yaodong Yang. VLA-arena: An open-source framework for benchmarking vision-language-action models. In *Proceedings of the 43rd International Conference on Machine Learning*, volume 306 of *Proceedings of Machine Learning Research*, pages 156567–156614. PMLR, 2026. URL https://proceedings.mlr.press/v306/zhang26cp.html.

[91] Qizheng Zhang, Changran Hu, Shubhangi Upasani, Boyuan Ma, Fenglu Hong, Vamsidhar Kamanuru, Jay Rainton, Chen Wu, Mengmeng Ji, Hanchen Li, Urmish Thakker, James Y. Zou, and Kunle Olukotun. Agentic context engineering: Evolving contexts for self-improving language models. In *International Conference on Learning Representations*, pages 86069–86100, 2026.

[92] Wenbo Zhang, Kaixuan Wang, Yutao Ouyang, Xiaoyu Huang, Liyang Li, Kailun Su, Weiyang Jin, Wenhao Chai, Haotian Liang, Zhiyang Dou, Yue Chen, and Tianxing Chen. An unexpected robot policy: Early evaluations of GPT-6 Astra on RoboDojo and beyond. *arXiv preprint arXiv:2609.24170*, 2026.

[93] Yixian Zhang, Huanming Zhang, Feng Gao, Xiao Li, Zhihao Liu, Yi Nie, Chunyang Zhu, Jiaxing Qiu, Yuchen Yan, Jiyuan Liu, Wenhao Tang, Jiaji Rao, Zhengru Fang, Changxu Wei, Yu Wang, Wenbo Ding, and Chao Yu. Harness VLA: Steering frozen VLAs into reliable manipulation primitives via memory-guided agents. *arXiv preprint arXiv:2607.08448*, 2026.

[94] Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. ExpeL: LLM agents are experiential learners. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pages 19632–19642. Association for the Advancement of Artificial Intelligence, 2024. doi: 10.1609/aaai.v38i17.29936.

[95] Wenshuai Zhao, Jorge Peña Queralta, and Tomi Westerlund. Sim-to-real transfer in deep reinforcement learning for robotics: a survey. In *2020 IEEE Symposium Series on Computational Intelligence (SSCI)*, pages 737–744. IEEE, 2020. doi: 10.1109/SSCI47803.2020.9308468.

[96] Jinliang Zheng, Jianxiong Li, Zhihao Wang, Dongxiu Liu, Xirui Kang, Yuchun Feng, Yinan Zheng, Jiayin Zou, Yilun Chen, Jia Zeng, Tai Wang, Ya-Qin Zhang, Jingjing Liu, and Xianyuan Zhan. X-VLA: Soft-prompted transformer as scalable cross-embodiment vision-language-action model. In *International Conference on Learning Representations*, volume 2026, pages 60580–60606, 2026. URL https://proceedings.iclr.cc/paper_files/paper/2026/hash/630d293833e09e1ecd892a898a20b074-Abstract-Conference.html.

[97] Jiaming Zhou, Qihang Zhang, Gangwei Xu, Cunxin Fan, Yujie Zhao, Ruilin Wang, Yiming Luo, Shuai Yang, Xing Zhu, Yujun Shen, Junwei Liang, and Yinghao Xu. Zero-WAM: In-context world-action modeling from human videos for open-ended task generalization. *arXiv preprint arXiv:2608.26103*, 2026.

[98] Xin Zhou, Dingkang Liang, Sifan Tu, Xiwu Chen, Yikang Ding, Dingyuan Zhang, Feiyang Tan, Hengshuang Zhao, and Xiang Bai. Hermes: A unified self-driving world model for simultaneous 3d scene understanding and generation. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2025.

[99] Xin Zhou, Dingkang Liang, Xiwu Chen, Feiyang Tan, Dingyuan Zhang, Hengshuang Zhao, and Xiang Bai. Hermes++: Toward a unified driving world model for 3d scene understanding and generation. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2026. doi: 10.1109/TPAMI.2026.3735606.

[100] Xin Zhou, Zongchuang Zhao, Zhibo Yang, Mingsheng Li, Humen Zhong, Shuai Bai, Du Chu, Ruizhe Chen, Zhaohai Li, Jun Tang, Qiuyue Wang, Mingkun Yang, Jiazhao Zhang, Dayiheng Liu, Dingkang Liang, and Xiang Bai. Qwen-drive-1.0: An initial step towards a vision-language foundation model for autonomous driving, 2026. URL https://arxiv.org/abs/2609.00111.

[101] Xueyang Zhou, Yangming Xu, Guiyao Tie, Yongchao Chen, Guowen Zhang, Duanfeng Chu, Pan Zhou, and Lichao Sun. LIBERO-PRO: Towards robust and fair evaluation of vision-language-action models beyond memorization. *arXiv preprint arXiv:2510.03827*, 2025.

[102] Zhiyuan Zhou, Pranav Atreya, Abraham Lee, Homer Rich Walke, Oier Mees, and Sergey Levine. Autonomous improvement of instruction following skills via foundation models. In Pulkit Agrawal, Oliver Kroemer, and Wolfram Burgard, editors, *Proceedings of The 8th Conference on Robot Learning*, volume 270 of *Proceedings of Machine Learning Research*, pages 4805–4825. PMLR, 2025.

[103] Fangqi Zhu, Zhengyang Yan, Zicong Hong, Quanxin Shou, Xiao Ma, and Song Guo. Wmpo: World model-based policy optimization for vision-language-action models. In *International Conference on Learning Representations*, volume 2026, pages 62486–62502, 2026.

[104] Brianna Zitkovich, Tianhe Yu, Sichun Xu, Peng Xu, Ted Xiao, Fei Xia, Jialin Wu, Paul Wohlhart, Stefan Welker, Ayzaan Wahid, Quan Vuong, Vincent Vanhoucke, Huong Tran, Radu Soricut, Anikait Singh, Jaspiar Singh, Pierre Sermanet, Pannag R. Sanketi, Grecia Salazar, Michael S. Ryoo, Krista Reymann, Kanishka Rao, Karl Pertsch, Igor Mordatch, Henryk Michalewski, Yao Lu, Sergey Levine, Lisa Lee, Tsang-Wei Edward Lee, Isabel Leal, Yuheng Kuang, Dmitry Kalashnikov, Ryan Julian, Nikhil J. Joshi, Alex Irpan, Brian Ichter, Jasmine Hsu, Alexander Herzog, Karol Hausman, Keerthana Gopalakrishnan, Chuyuan Fu, Pete Florence, Chelsea Finn, Kumar Avinava Dubey, Danny Driess, Tianli Ding, Krzysztof Marcin Choromanski, Xi Chen, Yevgen Chebotar, Justice Carbajal, Noah Brown, Anthony Brohan, Montserrat Gonzalez Arenas, and Kehang Han. RT-2: Vision-language-action models transfer web knowledge to robotic control. In Jie Tan, Marc Toussaint, and Kourosh Darvish, editors, *Proceedings of The 7th Conference on Robot Learning*, volume 229 of *Proceedings of Machine Learning Research*, pages 2165–2183. PMLR, 2023. URL https://proceedings.mlr.press/v229/zitkovich23a.html.